Shree Swami Atmanand Saraswati Institute of Technology
affiliated with
Gujrat Technological University, Ahmedabad

A

PROJECT REPORT SUBMITTED

ON

Micro-SIEM: A Lightweight Security Information and Event Management System

Prepared as a part of the requirements for the subject of

Minor Project (BC05001081)

Submitted By

Jay Sorathiya (243050601123)

Pratixa Rajodiya (243050601105)

Under the guidance of

Prof. Vipul K. Chaudhari

3rd Year, BCA, Semester - V

Bachelor of Computer Application Department

Shree Swami Atmanand Saraswati Institute of Technology

Department of Bachelor of Computer Application

Academic Year 2026-27

Shree Swami Atmanand Saraswati Institute of Technology
affiliated with
Gujrat Technological University, Ahmedabad

CERTIFICATE

This is to certify that the project report submitted along with the project entitled "**Micro-SIEM: A Lightweight Security Information and Event Management System**" has been carried out by **Jay Sorathiya** (enrollment no.: **243050601123) and Pratixa Rajodiya (enrollment no.: 243050601105)** under the guidance of **Prof. Vipul K. Chaudhari** as part of Project (BC05001081) 3rd Year (5th Semester) of BCA in the academic year 2026-2027 and their work is satisfactory.

Date: / / 2026

Place: SSASIT, Surat

Prof. Vipul K. Chaudhari
Internal Guide

Prof. Sanket Trivedi
Head of BCA Department

SHREE SWAMI ATMANAND SARASWATI INSTITUTE OF TECHNOLOGY

Self-Declaration

We hereby declare that the project report submitted along with the project entitled **Micro-SIEM: A Lightweight Security Information and Event Management System** submitted as part of Minor Project (BC05001081) 3rd Year (5th Semester) of **BCA** to Gujarat Technological University, Ahmedabad, is a bonafide record of original project work carried out by us and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

Jay Sorathiya
(243050601123)

Pratixa Rajodiya
(243050601105)

Acknowledgments

The success of any task relies on the efforts made by person but it cannot be achieved without cooperation of other persons which are being helpful. So, we would like to thank **SHREE SWAMI ATMANAND SARASWATI INSTITUTE OF TECHNOLOGY, SURAT**. for giving us the opportunity of doing this Project. We would like to take this opportunity to express our sincere thanks to all those people without whose support and co-operation, it would have been difficult to complete this project.

Primarily, we are very much thankful to our Internal Faculty guide **Prof. Vipul K. Chaudhari** for their leading guidance and sincere efforts throughout project work. They took deep interest in simplifying the difficulties. Also, they have been consistent source of inspiration for us.

We also express thanks to **Prof.Vaishnavi Bhathawala** and **Prof. Vipul Chaudhari,** class coordinator for her personal involvement, constructive suggestion and thoughtful idea for betterment of the project. We are grateful to our **H.O.D. Prof. Sanket Trivedi** and our beloved Principal **Dr. Jignesh Vaghasiya** for providing us deep knowledge and all necessary resources. We are also thankful to our Friends and Non-teaching Staff for their valuable time & help for completion project.

Once again, we are grateful to all those without whom this work would not have been successful.

Jay Sorathiya
(243050601123)

Pratixa Rajodiya
(243050601105)

INDEX

	Subject		Page No.
1	Project Description		
	1.1	Project Profile	4
	1.2	Project Definition	4
2	Environment Description		
	2.1	Hardware and Software Specification	6
	2.2	Development Description	7
3	System Description		
	3.1	Introduction	8
	3.2	Scope of the System	8
	3.3	Objective of the System	9
	3.4	Functionality of System	9
	3.5	Expected Outcome	11
4	Project Planning and Scheduling		
	4.1	Feasibility Study	12
	4.2	Software Engineering Model	13
	4.3	Requirement Analysis and Data Gathering	14
	4.4	Time Line Chart	16
5	System Analysis & Design		
	5.1	Use Case Diagram	17
	5.2	Class Diagram	19
	5.3	Activity Diagram	20
	5.4	Deployment Diagram	23
	5.5	Database Design	24
6	System Implementation		
	6.1	Source Code / Screen Shots	26
7	System Testing		40
8	Conclusion and Future Scope		44
9	Bibliography		46

1. PROJECT DESCRIPTION

➢ Introduction

Micro-SIEM is a lightweight **Security Information and Event Management (SIEM)** system built from scratch for learning, demonstration and small-scale monitoring. It continuously collects security-relevant log events from a web application and its Nginx web server, stores them centrally in OpenSearch, runs automated detection rules every few seconds, and presents the resulting alerts on role-based dashboards. Every alert is mapped to the **MITRE ATT&CK** framework (or to OWASP/CWE for web attacks), so an analyst immediately knows which attacker technique was observed.

The project also contains a deliberately weak **demo web portal** (login + search page) that acts as the attack target. Brute-force, password-spraying, credential-compromise and Cross-Site Scripting (XSS) attacks performed against this portal are captured as logs and detected live, which makes the complete SIEM life-cycle (collect, store, detect, alert, respond) visible in a single demonstration.

1) Existing System

Small organizations, college labs and students usually have no central place where security logs are analysed. The common situation has several problems:

- **Scattered logs** – Web server, application and authentication logs are stored as separate files on different machines and are rarely read.
- **Manual inspection** – An administrator must open log files and search them by hand (grep), which is slow and error-prone.
- **No real-time detection** – Attacks such as brute force or password spraying are discovered late, often only after an account is compromised.
- **No context** – Raw log lines do not say which attack technique they represent or how severe it is.
- **Commercial SIEMs are heavy** – Splunk, IBM QRadar and Microsoft Sentinel are powerful but expensive, licence-bound and need large hardware, which puts them out of reach for students and small teams.

➔ Modules in the Micro-SIEM System

- **Log Collection Module** – Demo web login events and Nginx access logs written as structured JSON lines.
- **Log Ingestion Module** – Vector tails the log files, parses them and ships them to OpenSearch.
- **Storage & Search Module** – OpenSearch indexes every event for fast queries and aggregations.
- **Detection Engine Module** – Four correlation/threshold rules run automatically every 5 seconds.
- **Alert Management Module** – De-duplicated, MITRE-mapped alerts are stored as a permanent history.

- **Dashboard Module** – Admin console and read-only viewer dashboard with live charts.
- **Authentication & Access Control Module** – Token sessions, admin/viewer roles and login rate limiting.
- **Demo Attack Surface Module** – Intentionally vulnerable login and search pages used as the attack target.
- **Response Module (future)** – IP firewall blocking and account blocking, built and tested but switched off.

2) Need for the New System

Micro-SIEM addresses the shortcomings of manual log handling by providing:

- **Centralized logging** – All events from different sources are stored in one searchable place.
- **Automated real-time detection** – Attacks are detected within seconds without anyone watching the logs.
- **Standard threat mapping** – Alerts carry MITRE ATT&CK technique IDs (T1110, T1110.003, T1078) or CWE-79.
- **Clear visualization** – Charts and tables show login success rate, log activity and attack vectors at a glance.
- **Role-based access** – Administrators can investigate and act; viewers can only observe.
- **Low cost** – Built only with free and open-source software, running on one laptop with Docker.
- **Learning value** – Every component (ingestion, storage, detection, dashboard) is small enough to be understood completely.

3) Objectives of the New System

1. **Centralize security logs** from more than one source into a single searchable store.
2. **Detect common credential and web attacks automatically** with transparent, explainable rules.
3. **Map every alert to an industry framework** (MITRE ATT&CK / OWASP CWE) with a severity level.
4. **Visualize security posture** through live dashboards for different user roles.
5. **Protect the SIEM itself** with authentication, authorization, rate limiting, input validation and output escaping.
6. **Run anywhere** using Docker containers started with a single command.

4) Problem Definition

- Security events are produced continuously, but without collection and correlation they are useless.
- An attacker trying many passwords (brute force) or one password against many accounts (spraying) must be noticed quickly.

- A successful login after several failures can mean a guessed credential and must be highlighted.
- Malicious input such as XSS payloads in URLs must be recognised even when it is URL-encoded.
- Alerts must not be duplicated every few seconds while an attack continues, otherwise analysts suffer alert fatigue.
- The monitoring system itself must not become a new attack surface (spoofed IPs, stored XSS, open database).

5) Core Components

➔ User Categories

The Micro-SIEM system has the following types of users:

1. Admin (Security Analyst)

- Logs in to the admin console with a session token.
- Views dashboard statistics, all raw logs and all alerts.
- Analyses an alert and opens the mitigation panel (IP / account blocking – planned).
- Views the blocked list and logs out.

2. Viewer

- Logs in to a read-only dashboard.
- Sees statistics and charts but cannot see raw logs or take any action (enforced on the server with HTTP 403).

3. Demo User / Attacker

- Uses the demo portal on port 8080: login page and search page.
- Every login attempt and every page request is logged and analysed.
- Simulates attacks: brute force, password spraying, credential compromise and XSS.

4. Detection Engine (system actor)

- A background task that wakes up every 5 seconds, runs all rules and writes new alerts.

1.1 PROJECT PROFILE

Project Attribute	Details
Project Title	Micro-SIEM: A Lightweight Security Information and Event Management System
Project Category	Cyber Security – Web-based Security Monitoring System
Time Duration	4 Months (July 2026 – October 2026)
Front-End	HTML5, Tailwind CSS 3.4, JavaScript, Chart.js 4.5
Back-End	Python 3.11, FastAPI 0.111, Uvicorn
Database / Storage	OpenSearch 3.8 (NoSQL document store and search engine)
Log Pipeline	Vector 0.59 (log collector and shipper)
Web Server / Proxy	Nginx 1.30
Deployment	Docker Engine and Docker Compose (6 containers)
Threat Framework	MITRE ATT&CK, OWASP Top 10 / CWE-79
Security Measures	Token sessions, role-based access, login rate limiting, trusted-proxy IP validation, output escaping, localhost-only database
College Guide	Prof. Vipul K. Chaudhari
Developed By	Jay Sorathiya (243050601123) Pratixa Rajodiya (243050601105)

1.2 PROJECT DEFINITION

Micro-SIEM is a containerised Security Information and Event Management system that collects web authentication and web server logs, stores them in OpenSearch, detects attacks with four automated rules, and displays MITRE-mapped alerts on secure, role-based dashboards. It is designed to be small enough to run on a single laptop yet complete enough to demonstrate the full monitoring life-cycle used by real Security Operations Centres (SOC).

➔ Key Aspects of the System:

1. Purpose:

- To provide one central place where security events from different sources can be searched and analysed.
- To detect password attacks and web injection attempts automatically and in near real time.
- To demonstrate, in a practical and explainable way, how a SIEM works internally.

2. Core Functionalities:

- **Log Ingestion:** Two live log sources (web login events and Nginx access log) shipped by Vector into OpenSearch.
- **Detection Rules:** Brute force (T1110), password spraying (T1110.003), fail-then-success correlation (T1078) and XSS signatures (CWE-79).

- **Alert History:** Every alert is stored permanently, de-duplicated to one alert per rule per source IP per 5 minutes.
- **Admin Console:** Statistics, login success rate, live activity chart, attack-vector chart, full log stream and threat list.
- **Viewer Dashboard:** Read-only statistics and charts for managers or auditors.
- **Security of the SIEM:** Server-side sessions, roles, rate limiting, IP validation, escaping and localhost-only storage.

2. ENVIRONMENT DESCRIPTION

The development and deployment of Micro-SIEM require a defined hardware and software environment. Because every component runs inside Docker containers, the same environment can be reproduced on any machine with Docker installed. The following sections describe the hardware and software specification and the development methodology.

2.1 HARDWARE AND SOFTWARE SPECIFICATION

A. Hardware Requirements

OpenSearch is the most resource-hungry component (it runs on the Java Virtual Machine with a 512 MB heap), so memory is the most important hardware requirement.

Component	Minimum Requirement	Recommended Requirement	Used for Development
Processor	Dual-core x86-64, virtualization enabled	Quad-core or better	AMD Ryzen 7 7435HS (12 vCPU)
RAM	4 GB	8 GB or higher	8 GB
Storage	20 GB free	40 GB SSD	38 GB virtual disk (SSD)
Network	Ethernet / Wi-Fi (LAN)	LAN + Internet for ngrok demo	Wi-Fi LAN
Peripherals	Keyboard, Mouse, Monitor	Second device (phone / PC) for live demo	Laptop + mobile phone

B. Software Requirements

Software Component	Technology / Tool Used
Operating System	Kali GNU/Linux Rolling (kernel 6.19) – any OS with Docker works
Containerization	Docker Engine 28.5, Docker Compose 2.40
Backend Language & Framework	Python 3.11, FastAPI 0.111.0, Uvicorn 0.30.1, Pydantic 2.7.4
Database / Search Engine	OpenSearch 3.8.0 (opensearch-py 2.6.0 client)
Data Exploration UI	OpenSearch Dashboards 3.8.0
Log Shipper	Vector 0.59.0 (VRL transforms)
Web Server / Reverse Proxy	Nginx 1.30.6 (with real-IP module)
Frontend	HTML5, Tailwind CSS 3.4.17, JavaScript (ES2020), Chart.js 4.5.1
Browser	Chromium / Firefox
IDE / Code Editor	Visual Studio Code
Version Control	Git and GitHub (GitHub CLI)
Remote Demo Access	ngrok (HTTP tunnel)
Testing Tools	curl, headless Chromium, Python test scripts

2.2 DEVELOPMENT DESCRIPTION

Micro-SIEM was developed with a structured, modular approach so that each part of the pipeline could be built and verified independently before being connected to the next one.

A. Development Approach

The project followed an **Agile (iterative and incremental) methodology**. Work was divided into short sprints, each delivering a working, testable piece of the system. The key phases were:

1. **Requirement Analysis** – Studying SIEM concepts, MITRE ATT&CK techniques and the log formats to be collected.
2. **System Design** – Designing the container architecture, data flow, index structure and detection rules.
3. **Pipeline Development** – Setting up OpenSearch and Vector and verifying that log events reach the index.
4. **Backend & Detection Development** – Building the FastAPI service, demo login API, detection rules and alert storage.
5. **Dashboard Development** – Building the admin and viewer dashboards with authentication and role checks.
6. **Security Hardening** – Code review of the SIEM itself and fixing issues (IP spoofing, stored XSS, rate limiting, exposure).
7. **Testing & Documentation** – Unit, integration, live end-to-end and browser testing, followed by this report.

B. Modules & Development Tools

Module	Technology Used
Log Collection	FastAPI JSON-line logger, Nginx JSON access log format
Log Ingestion	Vector file sources, VRL parse_json transform, Elasticsearch-compatible bulk sink
Storage & Search	OpenSearch indices: logs-index, alerts-index, blocked-index
Detection Engine	Python, OpenSearch aggregations and scroll scan, asyncio background task
Alert Management	Deterministic document IDs + create() for de-duplication
Authentication & RBAC	FastAPI dependencies, secrets.token_hex session tokens, environment variables (.env)
Dashboards	HTML, Tailwind CSS, JavaScript fetch API, Chart.js
Reverse Proxy	Nginx proxy_pass /api/, real_ip module
Response (planned)	iptables / ip6tables (INPUT + DOCKER-USER chains)
Deployment	Docker Compose, named volumes, health checks

3. SYSTEM DESCRIPTION

3.1 INTRODUCTION

A Security Information and Event Management (SIEM) system combines two older ideas: **Security Information Management** (collecting and storing logs for analysis and compliance) and **Security Event Management** (real-time monitoring, correlation and alerting). Micro-SIEM implements both ideas on a small scale. Its working can be summarised as a five-stage pipeline:

1. **Generate** – The demo portal and Nginx write one JSON line per event.
2. **Collect & ship** – Vector reads new lines and sends them to OpenSearch in bulk.
3. **Store & index** – OpenSearch makes every event searchable within about a second.
4. **Detect** – The detection engine queries recent events every 5 seconds and evaluates four rules.
5. **Present & respond** – Alerts appear on the dashboards, where the admin can analyse them.

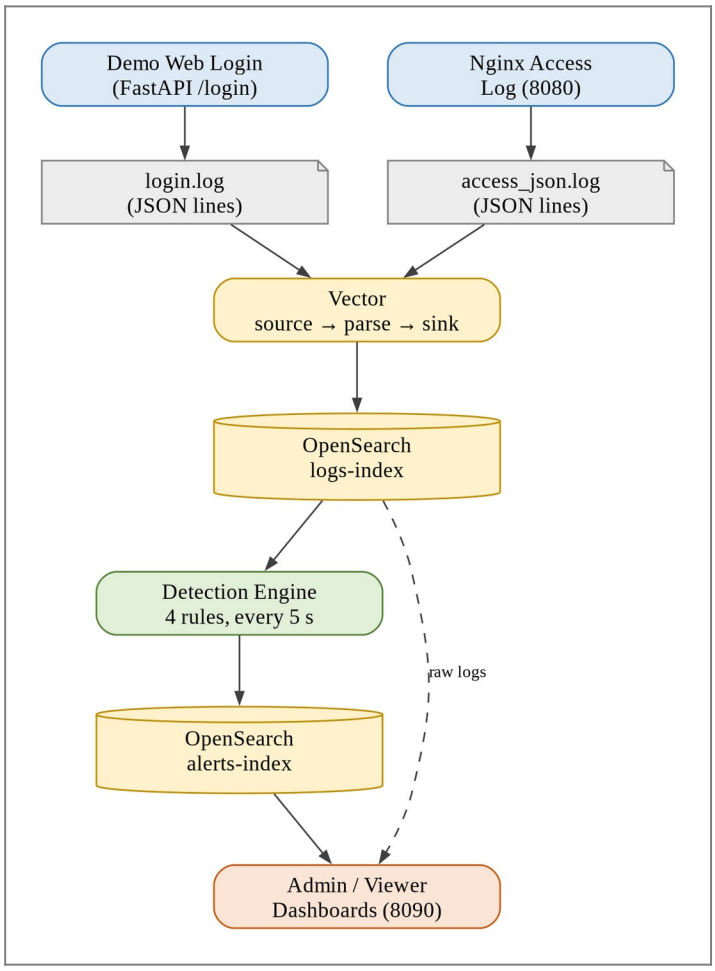


Figure 3.1: Micro-SIEM Data Flow Architecture

3.2 SCOPE OF THE SYSTEM

The scope defines what Micro-SIEM covers in this version and what is intentionally left out.

In Scope:

- Two live log sources: demo web-application login events and Nginx web server access logs (dashboard logins are also logged).
- Four detection rules: brute force, password spraying, fail-then-success correlation and XSS signature detection.
- Permanent, de-duplicated alert history with severity and MITRE ATT&CK / CWE mapping.
- Admin console and read-only viewer dashboard with server-side role enforcement.
- Deployment of the complete stack on one host with Docker Compose; access from LAN devices (demo portal also through ngrok).
- Security of the SIEM itself: authentication, rate limiting, IP trust validation, output escaping, localhost-only database.

Out of Scope (current version):

- Automatic blocking – the firewall and account blocking code exists but is disabled behind a "Coming Soon" message.
- Operating-system logs (SSH, Windows Event Logs), firewall logs and network packet capture.
- Machine-learning based anomaly detection and external threat-intelligence feeds.
- Multi-node clusters, long-term retention policies and cloud deployment.

3.3 OBJECTIVE OF THE SYSTEM

- **Visibility:** Give a single, real-time view of authentication activity and web traffic.
- **Fast detection:** Raise an alert within about 5–15 seconds of an attack pattern appearing.
- **Accuracy:** Keep different attacks separate (brute force vs. spraying) and avoid false positives on normal users.
- **Explainability:** Every alert states the rule that matched, the attempt count and the MITRE technique.
- **No alert fatigue:** One alert per rule per source IP per 5 minutes, even while an attack continues.
- **Secure by design:** The SIEM must not be usable to attack its own users (stored XSS, spoofed IPs, open database).
- **Easy deployment:** One command (docker compose up) starts the whole system.

3.4 FUNCTIONALITY OF SYSTEM

1. Log Collection

The FastAPI service writes every demo login attempt to **/var/log/webapp/login.log** as a JSON line containing the timestamp, log type, event (login_success, login_failed, login_blocked), username, source IP and result. Dashboard logins are recorded in the same file

with log type **dashboard_login**. Nginx writes every HTTP request to **access_json.log** using a custom JSON log format (timestamp, source IP, method, path, status).

2. Log Ingestion and Storage

Vector watches both files, parses each JSON line with a VRL **parse_json** transform and ships the events to the **logs-index** in OpenSearch with the bulk API. Vector remembers its read position in a persistent volume, so no lines are skipped or duplicated when the container restarts.

3. Detection Engine

The detection engine is a background task started together with the API. Every 5 seconds it runs four rules. Counting is done by OpenSearch aggregations, so large attacks are counted completely.

Rule	Logic	Window	Mapping
Web Login Brute Force	5 or more failed logins on the SAME username from one source IP (attempts on blocked accounts also count)	2 min	MITRE T1110
Password Spraying Attack	Failed logins on 3 or more DIFFERENT usernames from one source IP (case-insensitive)	5 min	MITRE T1110.003
Suspicious Login: Failed Attempts Then Success	2 or more failures followed by a successful login from the same source IP	2 min	MITRE T1078
XSS Attempt Detected	Known XSS signatures (<script, onerror=, javascript:, <img, <svg, …) in the URL-decoded request path	2 min	OWASP / CWE-79

4. Alert Management

Before writing an alert the engine checks whether the same alert type was raised for the same source IP in the last 5 minutes. The alert document ID is built from the alert type, source IP and 5-minute time window and written with OpenSearch **create()**, which refuses duplicates even if two detection runs overlap. Each alert stores the timestamp, alert type, source IP, attempt count, severity, matched rule and the MITRE technique, ID and tactic.

5. Admin Console

- Statistic cards: total logs, active alerts, suspicious IPs and login success rate.
- Charts: login authentication status (doughnut), log activity per minute (line) and attack vectors (colour-coded bar chart).
- All Logs view: the latest 100 events with readable details (e.g. "login_failed · user: admin" or "GET /search.html?q=… → 200").
- Threats & Action view: latest alert per source IP with severity, MITRE ID and an Analyze / Mitigate button.
- Blocked panel and mitigation console (blocking shows "Coming Soon" in this version).

6. Viewer Dashboard

- Same statistics and charts as the admin console, without raw logs, threat details or action buttons.
- Calls to admin-only endpoints are rejected by the server with HTTP 403.

7. Authentication and Access Control

- Dashboard login issues a random 48-character session token valid for 8 hours; every API call must send it as a Bearer token.
- Passwords are read from environment variables (.env file, not stored in the source code) and compared in constant time.
- After 5 failed logins from one IP within 5 minutes the API answers HTTP 429 (rate limiting).
- Logout invalidates the token on the server immediately.

8. Demo Attack Surface

- Home page, login page with three demo accounts, and a search page that reflects the query without escaping (intentionally vulnerable).
- Used to generate realistic attack traffic during the demonstration.

9. Response Module (planned, disabled)

The API contains complete code for blocking an IP at the host firewall (iptables / ip6tables in both the INPUT and DOCKER-USER chains) and for blocking a demo account. Protected addresses (loopback, the machine's own IP and the Docker network) can never be blocked. The feature is switched off with **BLOCKING_ENABLED = False** and is planned for a future version.

3.5 EXPECTED OUTCOME

- Every login attempt and web request is visible in one place within seconds.
- Each of the four attack types produces exactly one correctly classified, MITRE-mapped alert, while normal users produce none.
- Detection works continuously, even when no dashboard is open.
- Administrators and viewers see role-appropriate information; unauthorized actions are refused by the server.
- The SIEM resists attacks against itself: spoofed IP headers, stored XSS through log data and brute force on the dashboard login.
- The whole system starts with one command and works offline (all dashboard libraries are served locally).

4. PROJECT PLANNING AND SCHEDULING

4.1 FEASIBILITY STUDY

A feasibility study was carried out to check whether Micro-SIEM can be built and used successfully within the available time, budget and skills. Four aspects were evaluated.

A. Technical Feasibility

Technical feasibility checks whether the system can be built with the available technology and knowledge.

- **Evaluation of technologies**
  - OpenSearch provides indexing, search and aggregations needed for detection (open-source fork of Elasticsearch).
  - Vector is a fast, lightweight log shipper with a built-in transformation language (VRL).
  - FastAPI offers automatic request validation and interactive API documentation.
  - Docker Compose runs all six services with one configuration file.
- **Scalability assessment**
  - Aggregation queries let OpenSearch count events, so rules keep working for large attacks.
  - More log sources can be added by adding a Vector source without changing the detection code structure.
- **Security implementation**
  - Token-based sessions, roles, rate limiting and output escaping are implemented with standard library features.
  - OpenSearch is bound to localhost so the unauthenticated database is not reachable from the network.

B. Economic Feasibility

Economic feasibility checks whether the project is financially viable.

- **Cost-benefit analysis**
  - All software is free and open-source: OpenSearch (Apache 2.0), Vector (MPL 2.0), FastAPI (MIT), Nginx, Docker.
  - No licence fees, unlike commercial SIEMs that charge per GB of data ingested per day.
  - Runs on an existing laptop; no server or cloud cost.
- **Benefit**
  - Provides practical knowledge of SOC tools that are in high demand in the cyber-security industry.

- Can be reused by small labs or organizations as a starting point for log monitoring.

C. Operational Feasibility

Operational feasibility checks whether the system will be usable and accepted by its users.

- **System usability**
  - Dark, terminal-style dashboards that are readable on desktop and mobile screens.
  - Readable log details and colour-coded attack charts reduce analysis time.
- **Automation**
  - Detection, de-duplication and alert storage run automatically without operator action.
- **Accessibility**
  - Dashboards (port 8090) and demo portal (port 8080) are reachable from any device on the LAN, each through a single port; the demo portal was also exposed through ngrok.

D. Schedule Feasibility

Schedule feasibility ensures that the project can be completed on time.

- **Timeline evaluation**
  - The project followed a four-month plan (July – October 2026) divided into phases and sprints.
  - Agile sprints delivered a working pipeline early, reducing integration risk.
- **Risk mitigation strategies**
  - Image versions pinned to avoid surprise breaking upgrades before the demonstration.
  - Regular testing after each sprint and a written progress log (PROGRESS.md).

4.2 SOFTWARE ENGINEERING MODEL

A. Overview of the Agile Model

Micro-SIEM was developed using the **Agile (iterative and incremental) model**. Agile suits this project because the system is a pipeline of independent components: each sprint could add one component, test it end-to-end with the components already built, and adjust the plan based on what was learned (for example, adding the password-spraying rule and the security hardening sprint after code review).

➔ Iterative Development Process

- Each feature (log source, rule, dashboard, security fix) was built in a short sprint and tested immediately.
- Every sprint followed the cycle: plan → design → develop → test → review.

- Problems found in testing (e.g. duplicate alerts, "Invalid Date" timestamps, spoofable IP header) were fed back into the next sprint.
- Working software was available after every sprint, so the demonstration could be rehearsed early.

B. Sprints of the Project

Sprint	Goal	Deliverable
1	Infrastructure	Docker Compose stack: OpenSearch, OpenSearch Dashboards, Vector; first events indexed
2	Backend + demo portal	FastAPI service, demo login/search pages, Nginx JSON access log
3	Detection	Brute force, correlation and XSS rules; alert index with MITRE mapping; later password spraying
4	Dashboards + auth	Admin and viewer dashboards, token sessions, role-based access control
5	Response	IP and account blocking (later disabled behind a Coming Soon popup)
6	Hardening	Trusted-proxy IP handling, escaping, rate limiting, background detection, aggregation queries, pinned versions

C. Advantages of the Agile Model for this Project

- Early working pipeline reduced the risk of late integration failures.
- Flexible scope: new rules and fixes were added when their need became clear.
- Continuous testing produced a reliable system for the live demonstration.

4.3 REQUIREMENT ANALYSIS AND DATA GATHERING

A. Data Gathering Techniques

- **Study of documentation:** OpenSearch, Vector, FastAPI, Nginx and Docker official documentation.
- **Study of threat frameworks:** MITRE ATT&CK techniques T1110, T1110.003 and T1078; OWASP Top 10 and CWE-79.
- **Observation of real log formats:** Nginx access logs and authentication logs were studied to decide which fields are needed.
- **Review of existing SIEMs:** Concepts such as correlation rules, alert de-duplication and role-based dashboards were studied from Splunk, Wazuh and Elastic Security.
- **Discussions with the internal guide** to fix the scope, the rules and the demonstration flow.

B. Functional Requirements

ID	Requirement
FR-1	The system shall record every demo login attempt with timestamp, username, source IP and result.

ID	Requirement
FR-2	The system shall record every HTTP request to the demo portal (method, path, status, source IP).
FR-3	The system shall ship all events to a central searchable store within seconds.
FR-4	The system shall detect brute force, password spraying, fail-then-success and XSS attempts automatically.
FR-5	The system shall store alerts permanently with severity and MITRE ATT&CK / CWE mapping.
FR-6	The system shall raise at most one alert per rule per source IP per 5 minutes.
FR-7	The system shall authenticate dashboard users and support admin and viewer roles.
FR-8	The admin shall be able to view statistics, raw logs, alerts and the blocked list.
FR-9	The viewer shall only see statistics and charts.
FR-10	The system shall provide an API with interactive documentation.

C. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	Detection runs every 5 s; an alert appears within about 15 s of the attack.
NFR-2	Security	Server-side sessions, role checks, rate limiting, validated client IPs, escaped output, localhost-only database.
NFR-3	Reliability	Detection survives OpenSearch restarts; no duplicate alerts; persistent Vector checkpoints.
NFR-4	Usability	Readable dashboards in IST time, usable on desktop and mobile.
NFR-5	Portability	Runs on any OS with Docker; pinned versions; works offline.
NFR-6	Maintainability	Modular rules, documented code, progress log and README.

4.4 TIME LINE CHART

The project was planned over four months. The Gantt chart below shows the phases and sprints.

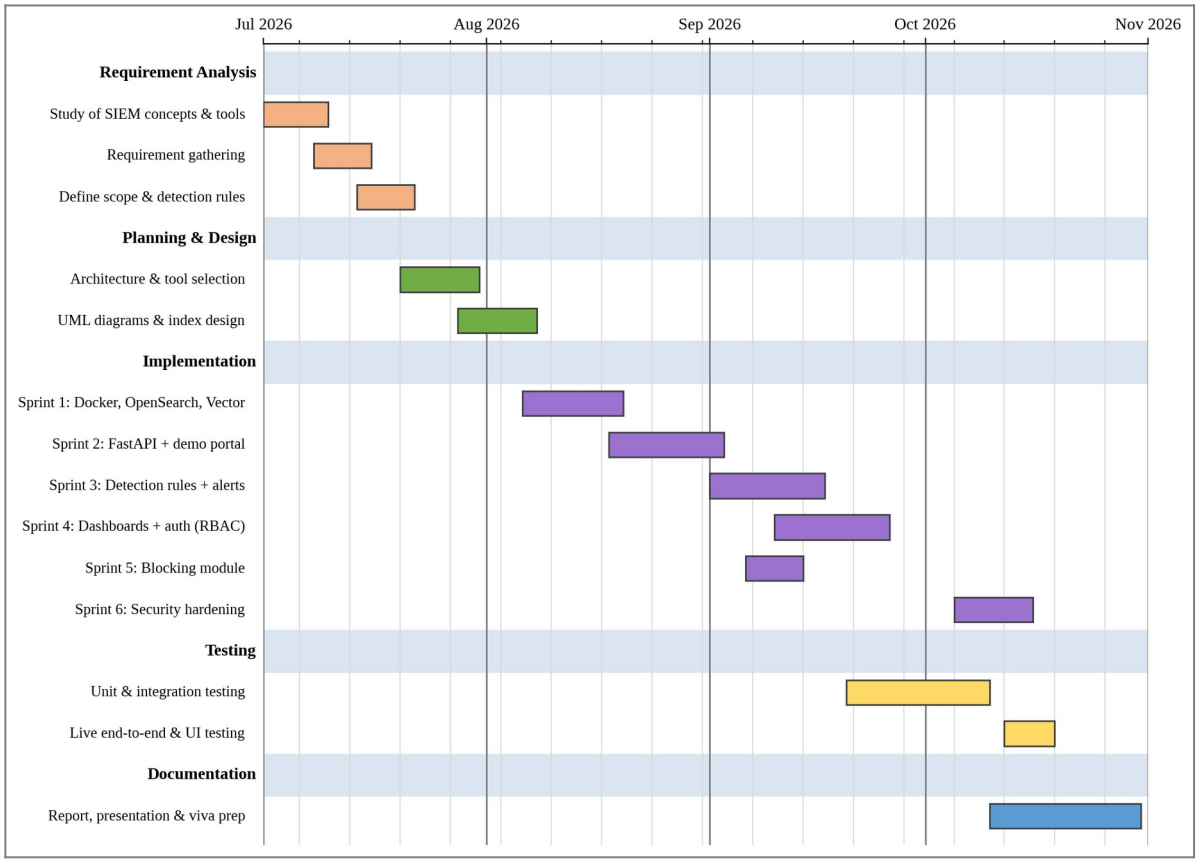


Figure 4.1: Project Time Line (Gantt) Chart

5. SYSTEM ANALYSIS & DESIGN

5.1 USE CASE DIAGRAM

The use case diagram shows the actors of Micro-SIEM and the functions each actor can perform.

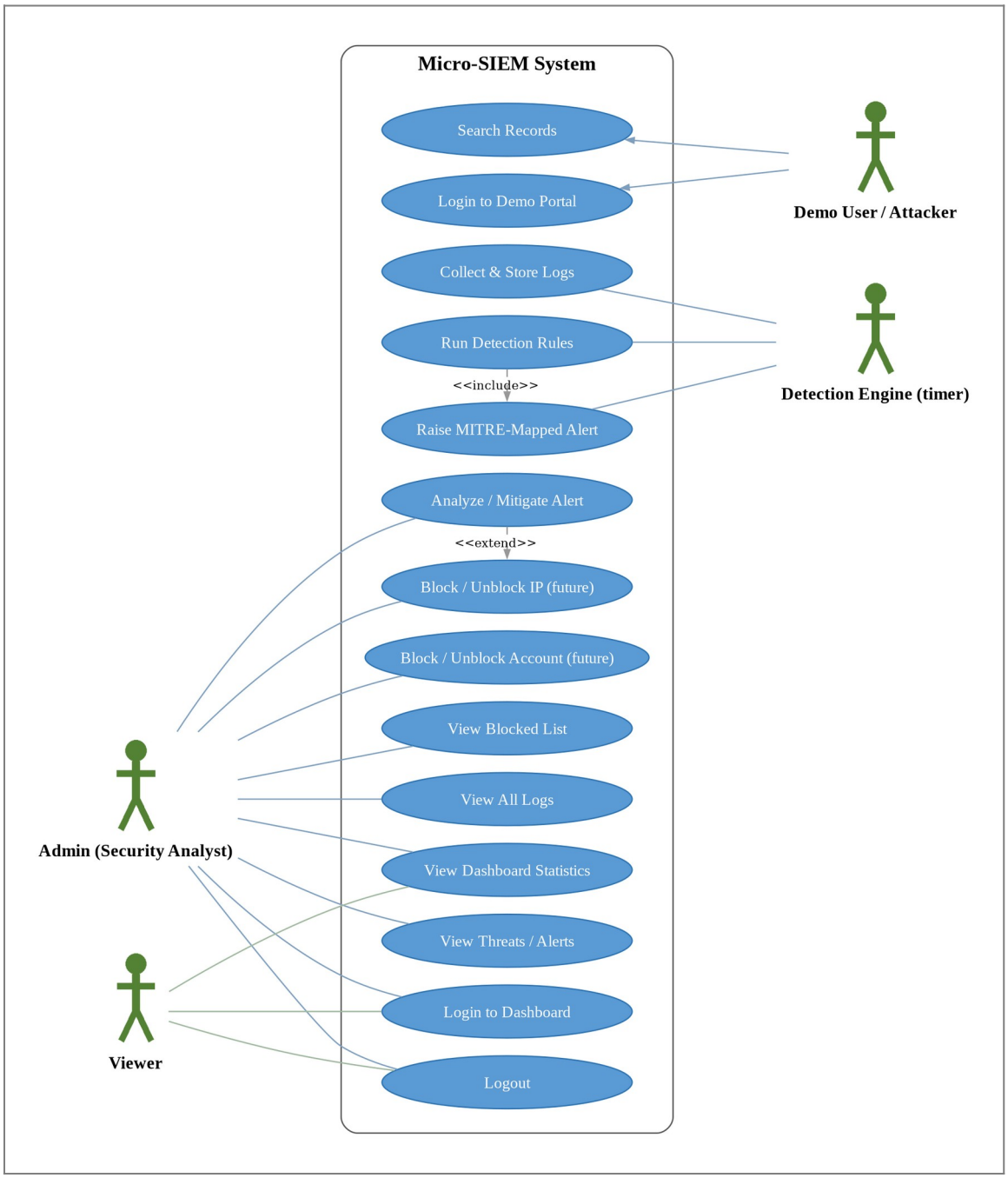


Figure 5.1: Use Case Diagram of Micro-SIEM

Use Case Description

Use Case	Actor	Description
Login to Dashboard	Admin, Viewer	Enter username and password; receive a session token and role (rate-limited).

Use Case	Actor	Description
View Dashboard Statistics	Admin, Viewer	See total logs, active alerts, suspicious IPs, success rate and charts.
View All Logs	Admin	See the latest events from all sources with readable details.
View Threats / Alerts	Admin	See the latest alert per source IP with severity and MITRE ID.
Analyze / Mitigate Alert	Admin	Open the security action panel for a source IP.
Block / Unblock IP or Account	Admin	Planned response action (disabled in this version).
View Blocked List	Admin	See blocked IPs and accounts.
Logout	Admin, Viewer	Invalidate the session token on the server.
Login to Demo Portal	Demo User / Attacker	Attempt a login; every attempt is logged.
Search Records	Demo User / Attacker	Search page that reflects the query (XSS target).
Collect & Store Logs	Detection Engine	Vector ships events to OpenSearch.
Run Detection Rules	Detection Engine	Evaluate the four rules every 5 seconds.
Raise MITRE-Mapped Alert	Detection Engine	Write a de-duplicated alert to the alert history.

5.2 CLASS DIAGRAM

The backend is a single FastAPI service. The class diagram shows its logical components: the API layer, the session manager, the detection engine with its four rules, the firewall manager and the data objects stored in OpenSearch.

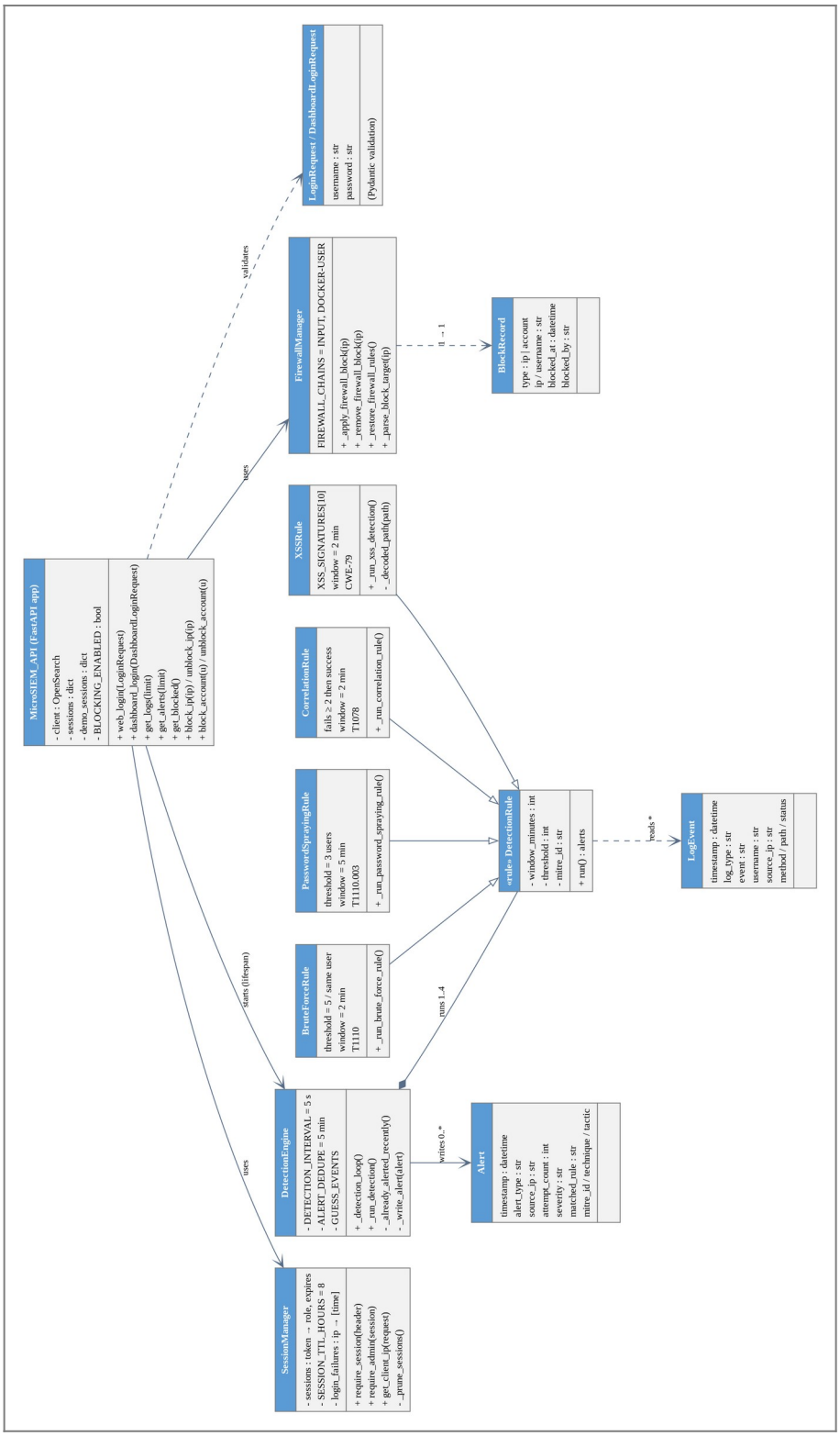


Figure 5.2: Class Diagram of the Micro-SIEM Backend (rotated)

5.3 ACTIVITY DIAGRAM

5.3.1 Dashboard Login and Admin Activity

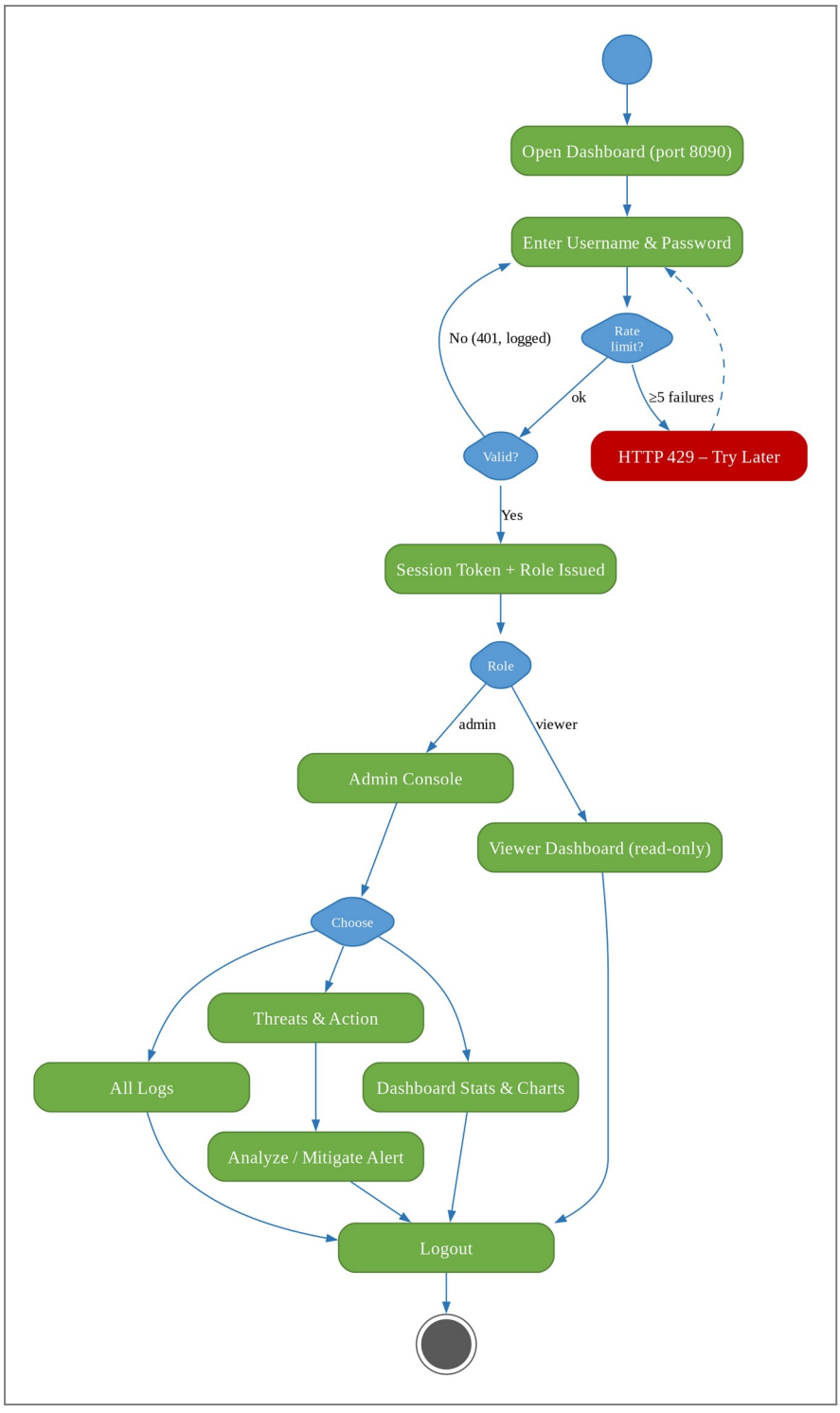


Figure 5.3: Activity Diagram – Dashboard Login and Admin Console

5.3.2 Detection Engine Activity

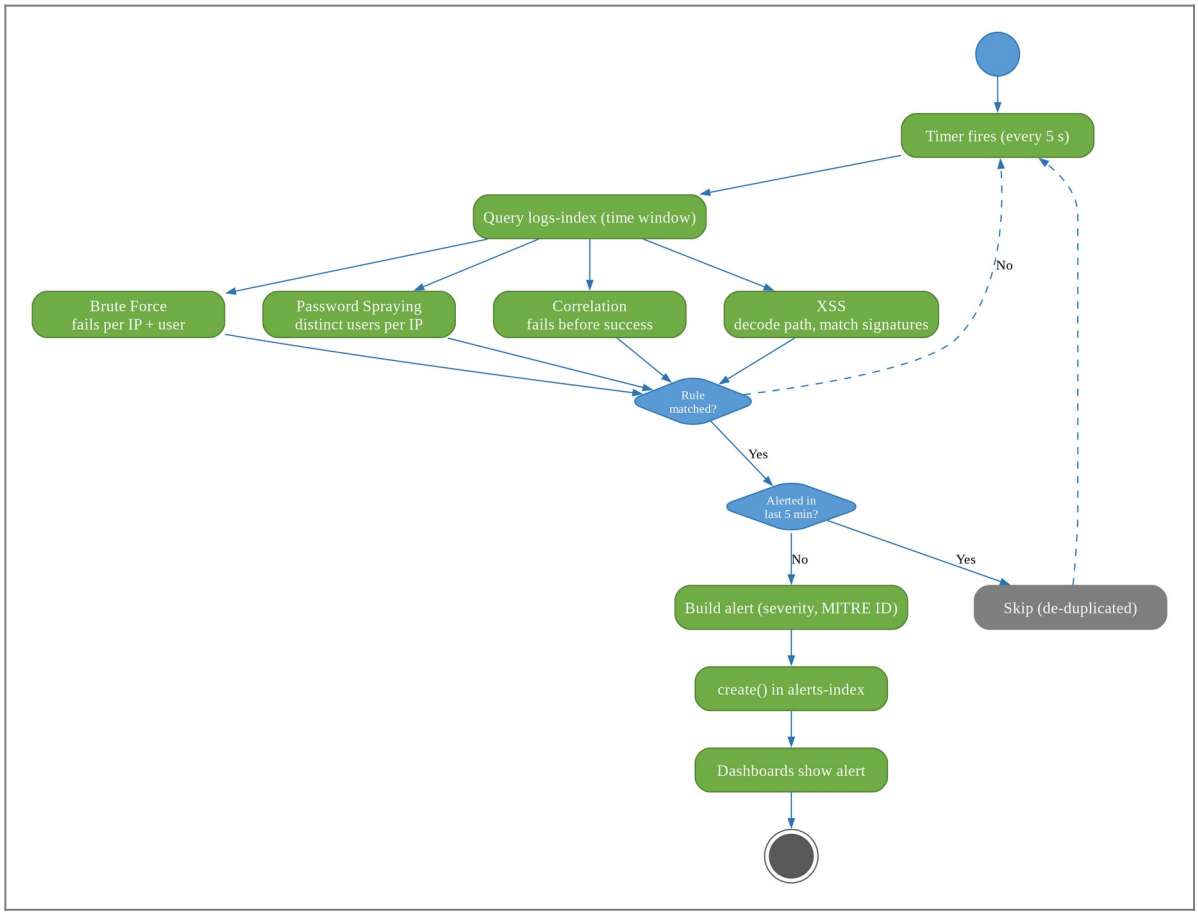


Figure 5.4: Activity Diagram – Detection Engine Cycle

5.3.3 Demo User / Attacker Activity

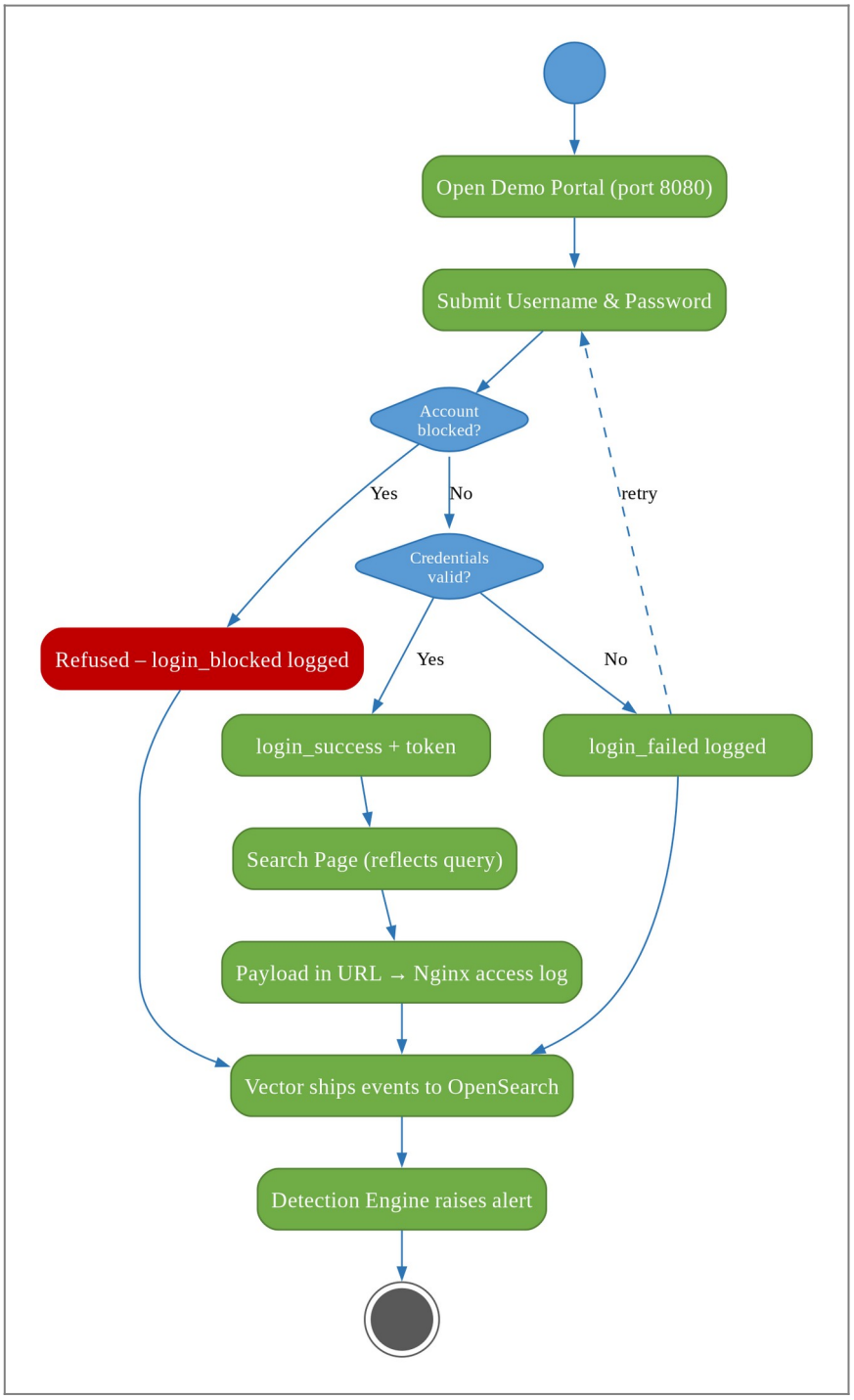


Figure 5.5: Activity Diagram – Demo Portal Login and Attack Flow

5.4 DEPLOYMENT DIAGRAM

Micro-SIEM is deployed as six Docker containers on one host. Only the demo portal (8080), the dashboards (8090) and the API (8000) are reachable from the network; OpenSearch and OpenSearch Dashboards are bound to 127.0.0.1.

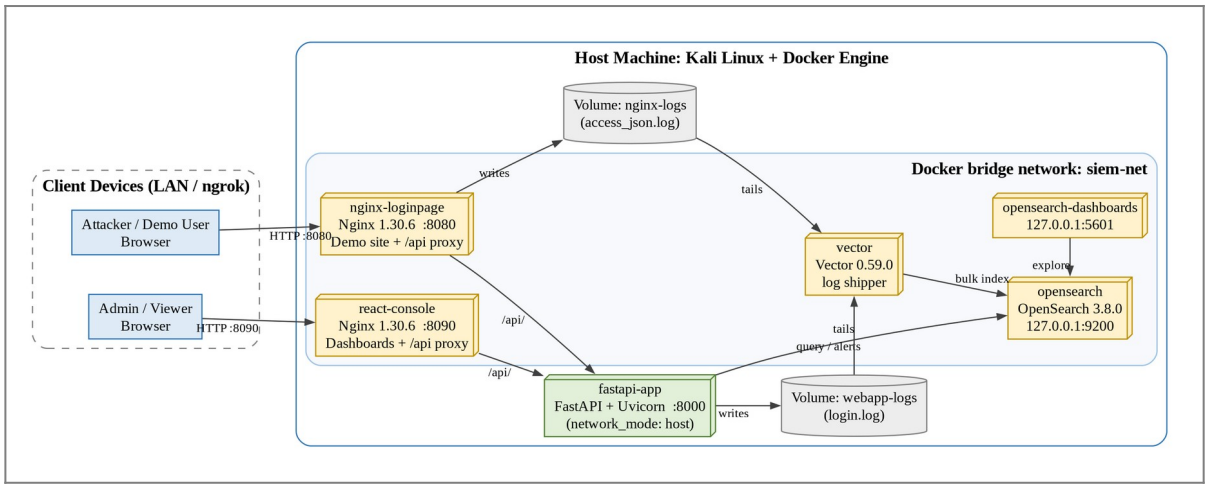


Figure 5.6: Deployment Diagram of Micro-SIEM

Container	Image	Port	Role
opensearch	opensearchproject/opensearch:3.8.0	127.0.0.1:9200	Stores and indexes logs, alerts, blocklist
opensearch-dashboards	opensearch-dashboards:3.8.0	127.0.0.1:5601	Raw data exploration
vector	timberio/vector:0.59.0-alpine	–	Tails log files, ships to OpenSearch
fastapi-app	micro-siem-api (Python 3.11)	8000 (host)	API, auth, detection engine
nginx-loginpage	nginx:1.30.6-alpine	8080	Demo portal + /api proxy
react-console	nginx:1.30.6-alpine	8090	Dashboards + /api proxy

5.5 DATABASE DESIGN

Micro-SIEM uses **OpenSearch**, a NoSQL document store. Data is stored as JSON documents in three indices (the equivalent of tables). Text fields are automatically indexed both as full-text (text) and as exact values (keyword) for filtering and aggregations. The structure below was read from the live index mappings.

1) Index: logs-index

Primary key: _id (generated by OpenSearch). Stores every event from both log sources.

Field Name	Data Type	Source	Description
timestamp	date	All	Time of the event (UTC)
log_type	text + keyword	All	web_login, dashboard_login or nginx_access
event	text + keyword	Login logs	login_success, login_failed, login_blocked, dashboard_login_*
username	text + keyword	Login logs	Username entered in the login form
source_ip	text + keyword	All	Validated client IP address
result	text + keyword	Login logs	success, failed or blocked
method	text + keyword	Nginx	HTTP method (GET, POST)
path	text + keyword	Nginx	Requested URI including the query string
status	long	Nginx	HTTP response status code
message	text + keyword	Vector	Original raw log line
file / host / source_type	text + keyword	Vector	Source file, collector host and source type metadata

2) Index: alerts-index

Primary key: _id = "alert_type | source_ip | 5-minute window" (deterministic, prevents duplicate alerts).

Field Name	Data Type	Description
timestamp	date	Time the alert was raised
alert_type	text + keyword	Name of the detected attack
source_ip	text + keyword	Attacker IP address
target_username	text + keyword	Targeted account (brute force only)
attempt_count	long	Number of attempts / distinct usernames / matching requests
severity	text + keyword	Severity level (HIGH)
matched_rule	text + keyword	Rule that matched, e.g. 5_failed_web_logins_same_user_2min
mitre_technique	text + keyword	Technique name, e.g. Brute Force
mitre_id	text + keyword	T1110, T1110.003, T1078 or CWE-79
mitre_tactic	text + keyword	Credential Access, Initial Access or N/A

3) Index: blocked-index

Primary key: _id (generated). Stores blocked IPs and accounts (used by the planned response module).

Field Name	Data Type	Description
type	text + keyword	ip or account
ip	text + keyword	Blocked IP address (normalised)
username	text + keyword	Blocked demo account
blocked_at	date	Time of blocking
blocked_by	text + keyword	Who blocked it (admin)

6. SYSTEM IMPLEMENTATION

6.1 SOURCE CODE / SCREEN SHOTS

6.1.1 Source Code

The most important parts of the implementation are shown below. The complete source code is available in the project repository (src/api/main.py, vector/vector.toml, nginx/, react-dashboard/ and docker-compose.yml).

A) docker-compose.yml – OpenSearch and API services (excerpt)

```
  opensearch:
    image: opensearchproject/opensearch:3.8.0
    container_name: opensearch
    environment:
      - discovery.type=single-node
      - DISABLE_SECURITY_PLUGIN=true      # simplifies local dev; do NOT do this in
production
      - "OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m"
    ports:
      - "127.0.0.1:9200:9200"   # REST API — this is what FastAPI/Vector talk to
      - "127.0.0.1:9600:9600"   # performance analyzer, not essential but harmless
    networks:
      - siem-net
    healthcheck:
      test: ["CMD-SHELL", "curl -s http://localhost:9200/_cluster/health || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 10

  api:
    build: ./src/api
    container_name: fastapi-app
    network_mode: "host"
    cap_add:
      - NET_ADMIN
    environment:
      - OPENSEARCH_HOST=localhost
      - OPENSEARCH_PORT=9200
      - DASHBOARD_ADMIN_PASSWORD=${DASHBOARD_ADMIN_PASSWORD:?set it in .env
(see .env.example)}
      - DASHBOARD_VIEWER_PASSWORD=${DASHBOARD_VIEWER_PASSWORD:?set it in .env
(see .env.example)}
    volumes:
      - webapp-logs:/var/log/webapp
    depends_on:
      opensearch:
        condition: service_healthy
```

B) vector/vector.toml – log ingestion pipeline

```
data_dir = "/var/lib/vector"

[sources.webapp_login]
type = "file"
include = ["/var/log/webapp/login.log"]
read_from = "end"          # only new lines from now on, not old history

[transforms.parse_webapp]
type = "remap"
inputs = ["webapp_login"]
source = '''
parsed, err = parse_json(.message)
if err == null {
    . = merge!(., parsed)
}
'''
```

```
[sources.nginx_access]
type = "file"
include = ["/var/log/nginx/access_json.log"]
read_from = "end"

[transforms.parse_nginx]
type = "remap"
inputs = ["nginx_access"]
source = '''
parsed, err = parse_json(.message)
if err == null {
    . = merge!(., parsed)
}
'''

[sinks.opensearch_logs]
type = "elasticsearch"
inputs = ["parse_webapp", "parse_nginx"]
endpoints = ["http://opensearch:9200"]
mode = "bulk"
api_version = "v8"

[sinks.opensearch_logs.bulk]
index = "logs-index"

[sinks.debug_console]
type = "console"
inputs = ["parse_webapp", "parse_nginx"]
encoding.codec = "json"
```

C) nginx/default.conf – JSON access log, trusted real IP and API proxy

```
log_format json_combined escape=json
  '{"timestamp":"$time_iso8601","source_ip":"$remote_addr",'
  '"method":"$request_method","path":"$request_uri",'
  '"status":$status,"log_type":"nginx_access"}';

server {
    listen 80;
    server_name localhost;
    access_log /var/log/nginx/access_json.log json_combined;
    set_real_ip_from 172.16.0.0/12;
    set_real_ip_from 127.0.0.1;
    real_ip_header X-Forwarded-For;
    real_ip_recursive on;
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
    location /api/ {
        proxy_pass http://host.docker.internal:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

D) main.py – validated client IP (prevents IP spoofing and stored XSS)

```
def get_client_ip(request: Request) -> str:
    peer = request.client.host if request.client else "unknown"
    if _in_networks(peer, TRUSTED_PROXY_NETWORKS):
        forwarded = _valid_ip(request.headers.get("x-real-ip", ""))
        if forwarded:
            return forwarded
    return _valid_ip(peer) or "unknown"
```

E) main.py – demo login endpoint (log source)

```
@app.post("/login")
def web_login(payload: LoginRequest, request: Request):
    source_ip = get_client_ip(request)
    username = payload.username
    now = datetime.utcnow()

    # Check if this account has been manually blocked by an admin
    if _account_blocked(username):
        write_webapp_log({
            "timestamp": now.isoformat(),
            "log_type": "web_login",
            "event": "login_blocked",
            "username": username,
            "source_ip": source_ip,
            "result": "blocked",
        })
        return {"success": False, "reason": "this account has been blocked by an
administrator"}

    if DEMO_ACCOUNTS.get(username) == payload.password:
        write_webapp_log({
            "timestamp": now.isoformat(),
            "log_type": "web_login",
            "event": "login_success",
            "username": username,
            "source_ip": source_ip,
            "result": "success",
        })
        _prune_sessions()
        demo_token = secrets.token_hex(16)
        demo_sessions[demo_token] = {"username": username, "expires": now +
timedelta(hours=SESSION_TTL_HOURS)}
        return {"success": True, "token": demo_token}

    write_webapp_log({
        "timestamp": now.isoformat(),
        "log_type": "web_login",
        "event": "login_failed",
        "username": username,
        "source_ip": source_ip,
        "result": "failed",
    })
    return {"success": False, "reason": "invalid credentials"}
```

F) main.py – background detection loop and de-duplicated alert writing

```
async def _detection_loop():
    while True:
        try:
            await asyncio.to_thread(_run_detection)
        except Exception:
            # e.g. OpenSearch still starting up - just try again next round
            logger.exception("Detection run failed")
        await asyncio.sleep(DETECTION_INTERVAL_SECONDS)

def _write_alert(alert: dict):
    window = int(datetime.utcnow().timestamp() // (ALERT_DEDUPE_MINUTES * 60))
    doc_id = f"{alert['alert_type']}|{alert['source_ip']}|{window}"
    try:
        client.create(index=ALERTS_INDEX, id=doc_id, body=alert, refresh="wait_for")
    except ConflictError:
        pass  # another detection run already wrote this alert
```

G) main.py – Brute force rule (MITRE T1110)

```
def _run_brute_force_rule():
    since = datetime.utcnow() - timedelta(minutes=TIME_WINDOW_MINUTES)
    query = {
```

```
        "size": 0,
        "query": _failed_logins_since(since, GUESS_EVENTS),
        "aggs": {
            "by_ip": {
                "terms": {"field": "source_ip.keyword", "size": AGG_BUCKET_LIMIT},
                "aggs": {
                    "by_user": {
                        "terms": {
                            "field": "username.keyword",
                            "size": AGG_BUCKET_LIMIT,
                            "min_doc_count": FAILED_LOGIN_THRESHOLD,
                        }
                    }
                },
            }
        },
    }
    result = client.search(index=LOGS_INDEX, body=query, ignore=[404])

    alert_type = "Web Login Brute Force"
    for ip_bucket in _buckets(result, "by_ip"):
        user_buckets = ip_bucket.get("by_user", {}).get("buckets", [])
        if not user_buckets:
            continue
        source_ip = ip_bucket["key"]
        if _already_alerted_recently(alert_type, source_ip):
            continue

        top = max(user_buckets, key=lambda b: b["doc_count"])
        alert = {
            "timestamp": datetime.utcnow().isoformat(),
            "alert_type": alert_type,
            "source_ip": source_ip,
            "target_username": top["key"],
            "attempt_count": top["doc_count"],
            "severity": "HIGH",
            "matched_rule": f"{FAILED_LOGIN_THRESHOLD}
_failed_web_logins_same_user_{TIME_WINDOW_MINUTES}min",
            "mitre_technique": "Brute Force",
            "mitre_id": "T1110",
            "mitre_tactic": "Credential Access",
        }
        _write_alert(alert)
```

H) main.py – Password spraying rule (MITRE T1110.003)

```
def _run_password_spraying_rule():
    since = datetime.utcnow() - timedelta(minutes=SPRAY_WINDOW_MINUTES)
    query = {
        "size": 0,
        "query": _failed_logins_since(since, GUESS_EVENTS),
        "aggs": {
            "by_ip": {
                "terms": {"field": "source_ip.keyword", "size": AGG_BUCKET_LIMIT},
                "aggs": {"by_user": {"terms": {"field": "username.keyword", "size":
AGG_BUCKET_LIMIT}}},
            }
        },
    }
    result = client.search(index=LOGS_INDEX, body=query, ignore=[404])

    alert_type = "Password Spraying Attack"
    for ip_bucket in _buckets(result, "by_ip"):
        # Case/whitespace variants of one name ("Admin", "admin ") count once
        usernames = {
            b["key"].strip().lower()
            for b in ip_bucket.get("by_user", {}).get("buckets", [])
            if b["key"].strip()
        }
        if len(usernames) < SPRAY_USERNAME_THRESHOLD:
            continue
        source_ip = ip_bucket["key"]
```

```
        if _already_alerted_recently(alert_type, source_ip):
            continue

        alert = {
            "timestamp": datetime.utcnow().isoformat(),
            "alert_type": alert_type,
            "source_ip": source_ip,
            "attempt_count": len(usernames),
            "severity": "HIGH",
            "matched_rule": f"{SPRAY_USERNAME_THRESHOLD}
_distinct_usernames_{SPRAY_WINDOW_MINUTES}min",
            "mitre_technique": "Password Spraying",
            "mitre_id": "T1110.003",
            "mitre_tactic": "Credential Access",
        }
        _write_alert(alert)
```

I) main.py – Fail-then-success correlation rule (MITRE T1078)

```
def _run_correlation_rule():
    since = datetime.utcnow() - timedelta(minutes=TIME_WINDOW_MINUTES)
    query = {
        "size": 0,
        "query": {
            "bool": {
                "filter": [
                    {"term": {"event.keyword": "login_success"}},
                    {"range": {"timestamp": {"gte": since.isoformat()}}},
                ]
            }
        },
        "aggs": {
            "by_ip": {
                "terms": {"field": "source_ip.keyword", "size": AGG_BUCKET_LIMIT},
                "aggs": {"last_success": {"max": {"field": "timestamp"}}},
            }
        },
    }
    result = client.search(index=LOGS_INDEX, body=query, ignore=[404])

    alert_type = "Suspicious Login: Failed Attempts Then Success"
    for ip_bucket in _buckets(result, "by_ip"):
        source_ip = ip_bucket["key"]
        last_success_ms = ip_bucket["last_success"]["value"]

        fail_query = _failed_logins_since(since)
        fail_query["bool"]["filter"] += [
            {"term": {"source_ip.keyword": source_ip}},
            {"range": {"timestamp": {"lt": int(last_success_ms), "format":
"epoch_millis"}}},
        ]
        fail_count = client.count(index=LOGS_INDEX, body={"query": fail_query},
ignore=[404]).get("count", 0)
        if fail_count < 2:
            continue
        if _already_alerted_recently(alert_type, source_ip):
            continue

        alert = {
            "timestamp": datetime.utcnow().isoformat(),
            "alert_type": alert_type,
            "source_ip": source_ip,
            "attempt_count": fail_count,
            "severity": "HIGH",
            "matched_rule": "correlation_fail_then_success",
            "mitre_technique": "Valid Accounts",
            "mitre_id": "T1078",
            "mitre_tactic": "Initial Access",
        }
        _write_alert(alert)
```

J) main.py – XSS signature rule (CWE-79)

```
def _decoded_path(path: str) -> str:
    decoded = path
    for _ in range(2):
        decoded = unquote_plus(decoded)
    return decoded.lower()

def _run_xss_detection():
    since = datetime.utcnow() - timedelta(minutes=TIME_WINDOW_MINUTES)
    query = {
        "query": {
            "bool": {
                "filter": [
                    {"term": {"log_type.keyword": "nginx_access"}},
                    {"range": {"timestamp": {"gte": since.isoformat()}}},
                ]
            }
        },
    }

    hits_by_ip = defaultdict(list)  # source_ip -> [matched signature per request]
    try:
        for hit in helpers.scan(client, index=LOGS_INDEX, query=query):
            event = hit["_source"]
            path = _decoded_path(event.get("path") or "")
            matched_sig = next((sig for sig in XSS_SIGNATURES if sig in path), None)
            if matched_sig:
                hits_by_ip[event.get("source_ip", "unknown")].append(matched_sig)
    except NotFoundError:
        return  # logs-index doesn't exist yet

    alert_type = "XSS Attempt Detected"
    for source_ip, sigs in hits_by_ip.items():
        if _already_alerted_recently(alert_type, source_ip):
            continue

        alert = {
            "timestamp": datetime.utcnow().isoformat(),
            "alert_type": alert_type,
            "source_ip": source_ip,
            "attempt_count": len(sigs),
            "severity": "HIGH",
            "matched_rule": f"xss_signature:{sigs[0]}",
            "mitre_technique": "N/A (OWASP Injection)",
            "mitre_id": "CWE-79",
            "mitre_tactic": "N/A",
        }
        _write_alert(alert)
```

K) main.py – dashboard login with rate limiting and logging

```
@app.post("/auth/login")
def dashboard_login(payload: DashboardLoginRequest, request: Request):
    source_ip = get_client_ip(request)
    now = datetime.utcnow()
    log_event = {
        "timestamp": now.isoformat(),
        "log_type": "dashboard_login",
        "username": payload.username,
        "source_ip": source_ip,
    }

    window_start = now - timedelta(minutes=DASHBOARD_LOGIN_WINDOW_MINUTES)
    recent = [t for t in dashboard_login_failures[source_ip] if t > window_start]
    dashboard_login_failures[source_ip] = recent
    if len(recent) >= DASHBOARD_LOGIN_MAX_FAILURES:
        write_webapp_log({**log_event, "event": "dashboard_login_rate_limited",
"result": "blocked"})
        raise HTTPException(status_code=429, detail="Too many failed attempts - try
again in a few minutes")
```

```
    user = DASHBOARD_USERS.get(payload.username)
    password_ok = bool(user and user["password"]) and secrets.compare_digest(
        user["password"].encode(), payload.password.encode()
    )
    if not password_ok:
        dashboard_login_failures[source_ip].append(now)
        write_webapp_log({**log_event, "event": "dashboard_login_failed", "result":
"failed"})
        raise HTTPException(status_code=401, detail="Invalid username or password")

    dashboard_login_failures.pop(source_ip, None)
    write_webapp_log({**log_event, "event": "dashboard_login_success", "result":
"success"})
    _prune_sessions()
    token = secrets.token_hex(24)
    sessions[token] = {
        "role": user["role"],
        "expires": now + timedelta(hours=SESSION_TTL_HOURS),
    }
    return {"token": token, "role": user["role"]}
```

L) main.py – session and role checks (RBAC)

```
def require_session(authorization: str = Header(None)):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Missing or malformed
Authorization header")
    token = authorization.split(" ", 1)[1]
    session = sessions.get(token)
    if not session:
        raise HTTPException(status_code=401, detail="Invalid session token")
    if datetime.utcnow() > session["expires"]:
        del sessions[token]
        raise HTTPException(status_code=401, detail="Session expired")
    return session

def require_admin(session: dict = Depends(require_session)):
    if session["role"] != "admin":
        raise HTTPException(status_code=403, detail="Admin role required for this
action")
    return session
```

M) admin.html – time conversion, readable log details and HTML escaping

```
function toIST(ts) {
    if (!ts) return '-';
    const hasZone = /([zZ]|[+-]\d\d:?\d\d)$/.test(ts);
    const d = new Date(hasZone ? ts : ts + 'Z');
    if (isNaN(d)) return '-';
    return d.toLocaleTimeString('en-GB', { timeZone: 'Asia/Kolkata', hour12: false });
}

function logDetail(l) {
    let text;
    if (l.log_type === 'nginx_access') {
        let path = l.path || '';
        try { path = decodeURIComponent(path.replace(/\+/g, ' ')); } catch (e) {}
        text = `${l.method || '?'} ${path} → ${l.status ?? '?'}`;
    } else if (l.event) {
        text = l.username !== undefined ? `${l.event} · user: ${l.username}` :
l.event;
    } else {
        text = (l.message || '').toString();
    }
    return text.length > 90 ? text.slice(0, 89) + '…' : text;
}

function escapeHtml(s) {
    return String(s ??
'').replace(/&/g,'&amp;').replace(/</g,'&lt;').replace(/>/g,'&gt;')
```

```
        .replace(/"/g,'&quot;').replace(/'/g,'&#39;');
}
```

6.1.2 Screen Shots

MODULE NAME – DEMO PORTAL (ATTACK TARGET)

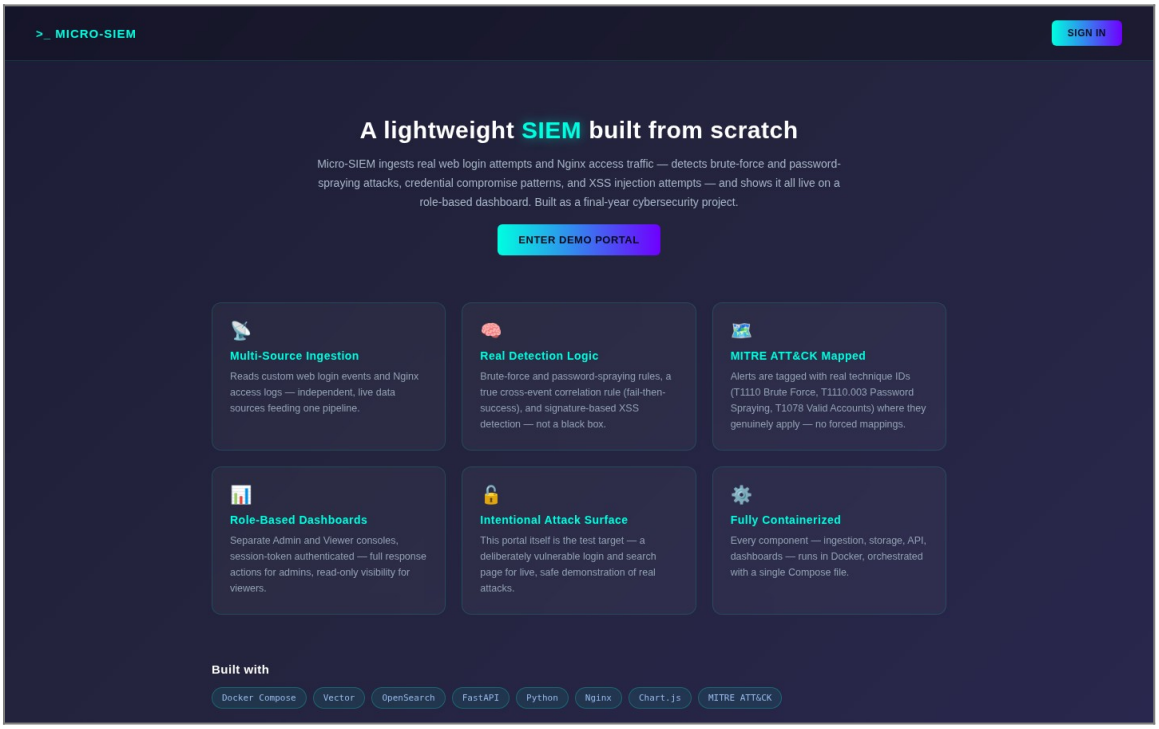


Figure 6.1: Demo Portal – Home Page (port 8080)

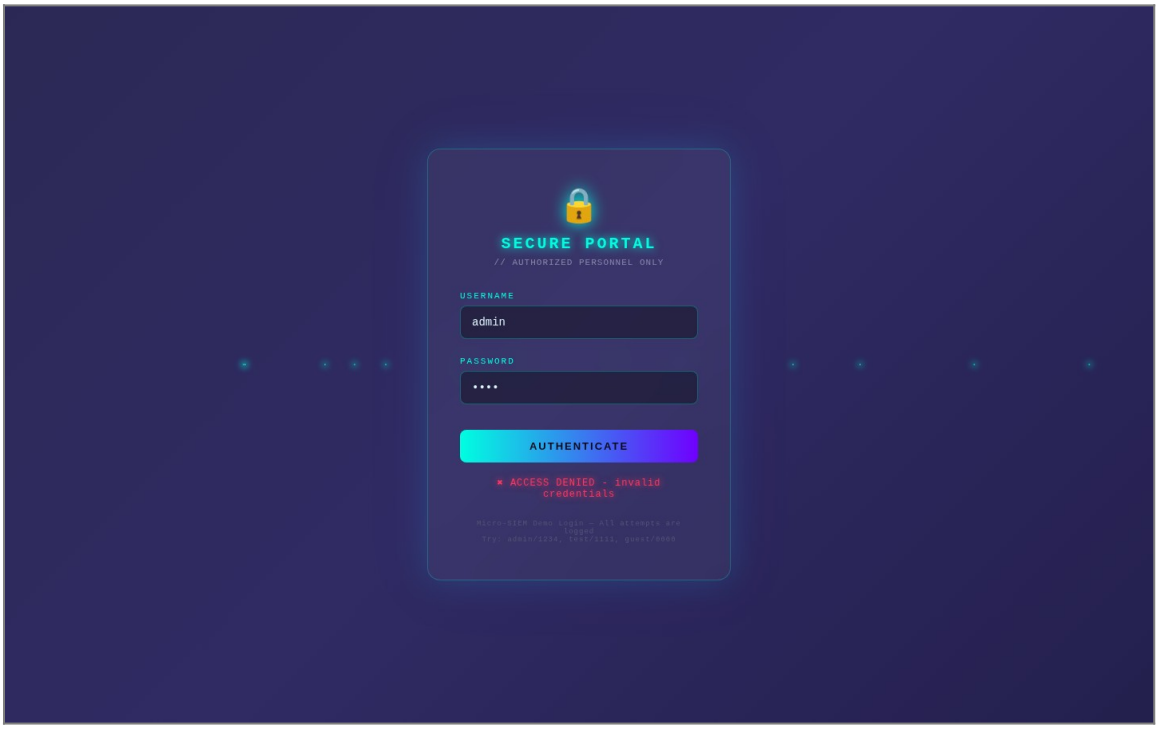


Figure 6.2: Demo Portal – Login Page showing a failed attempt (logged as login_failed)

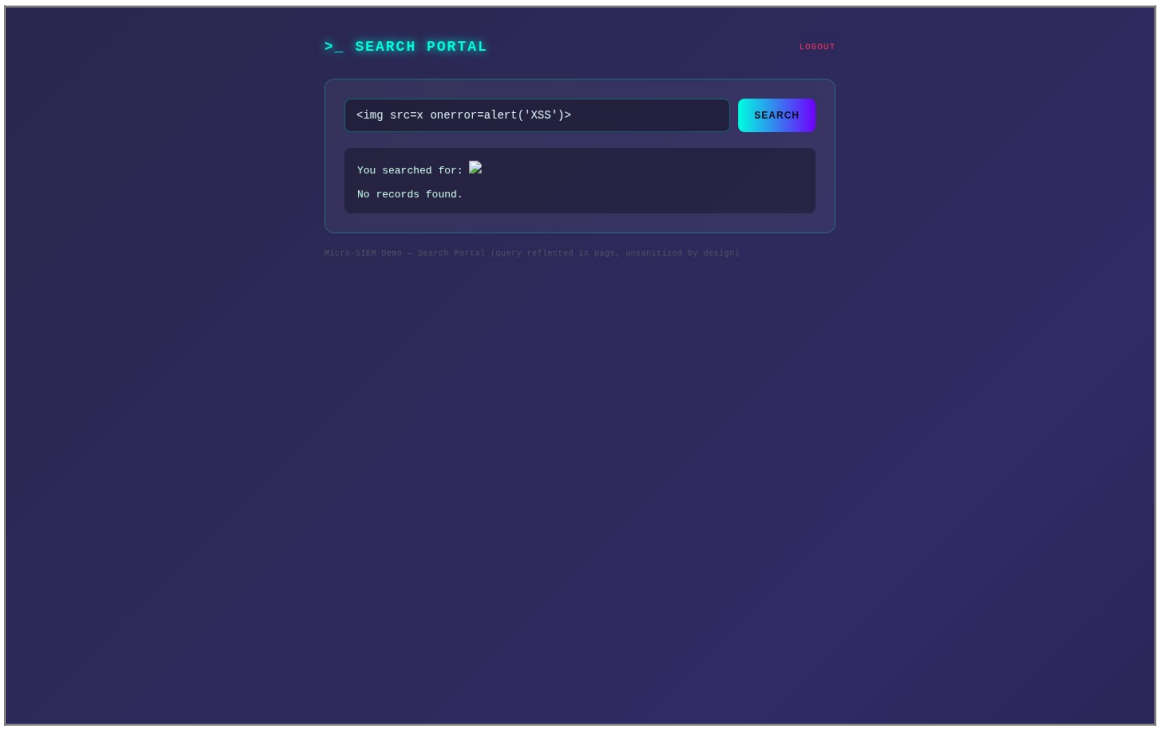


Figure 6.3: Demo Portal – Search Page reflecting an XSS payload (intentionally vulnerable)

MODULE NAME – DASHBOARD ACCESS

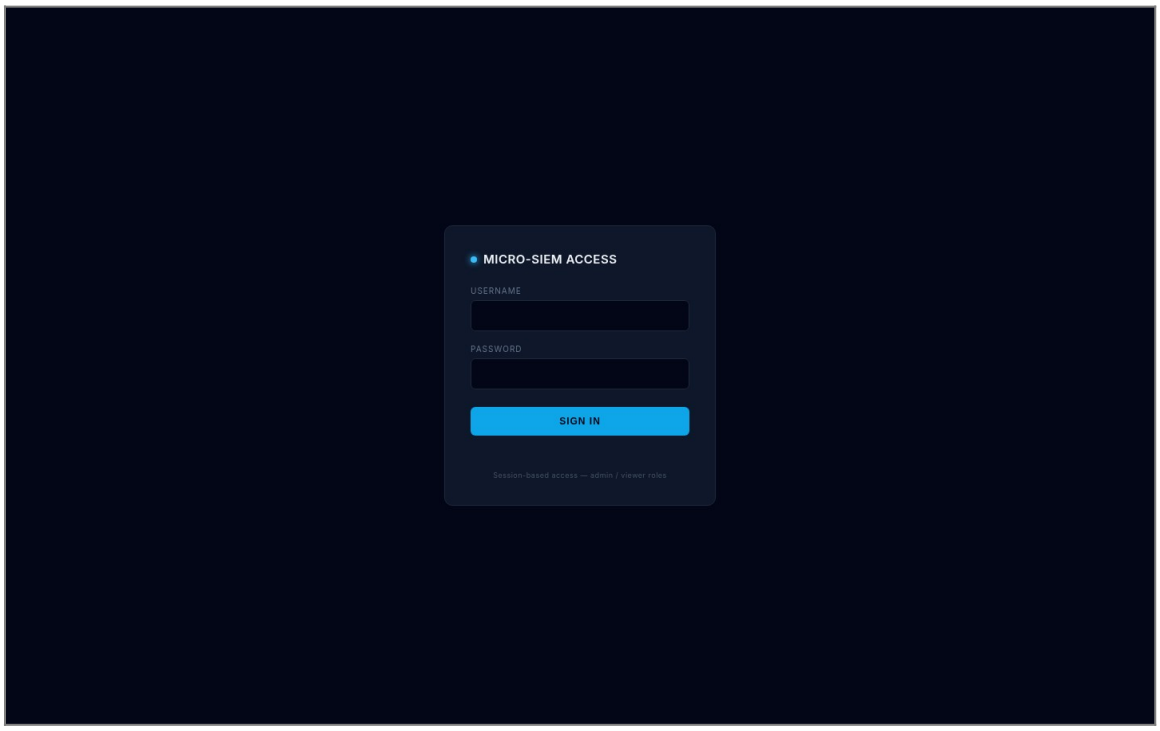


Figure 6.4: Micro-SIEM Dashboard Sign-In (port 8090)

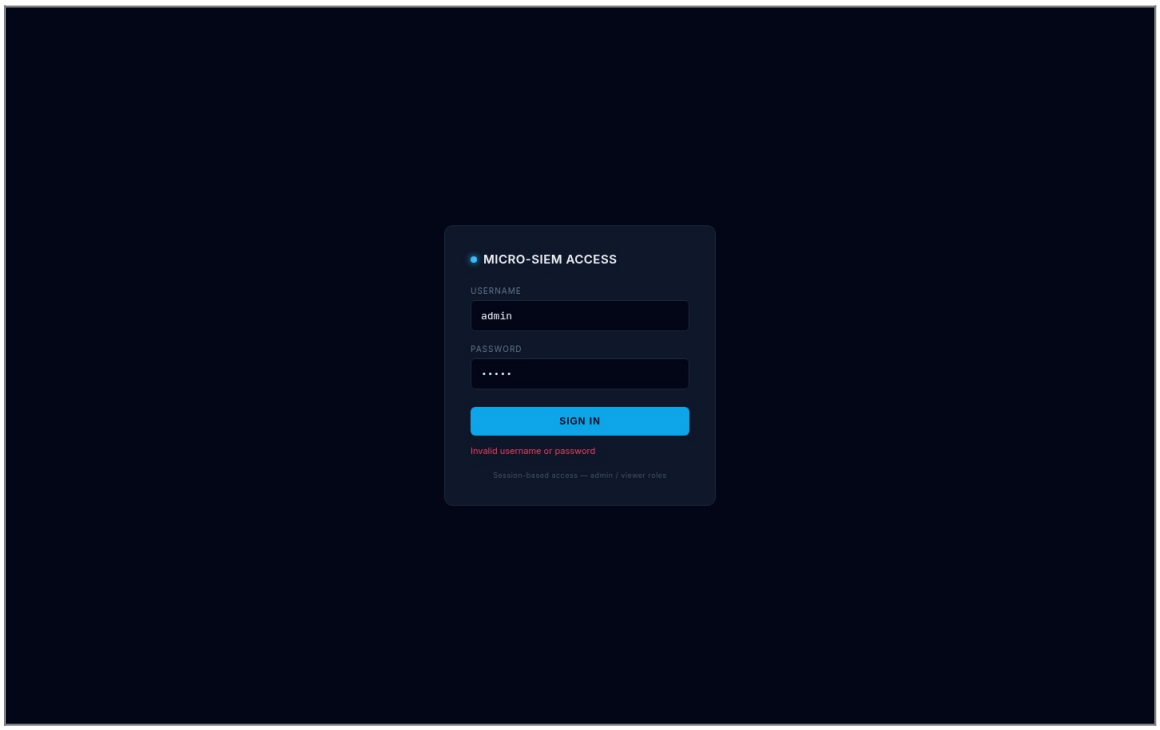


Figure 6.5: Dashboard Sign-In – invalid credentials (attempt logged, rate-limited after 5 failures)

MODULE NAME – ADMIN CONSOLE



Figure 6.6: Admin Console – Statistics, Login Status, Log Activity and Attack Vectors

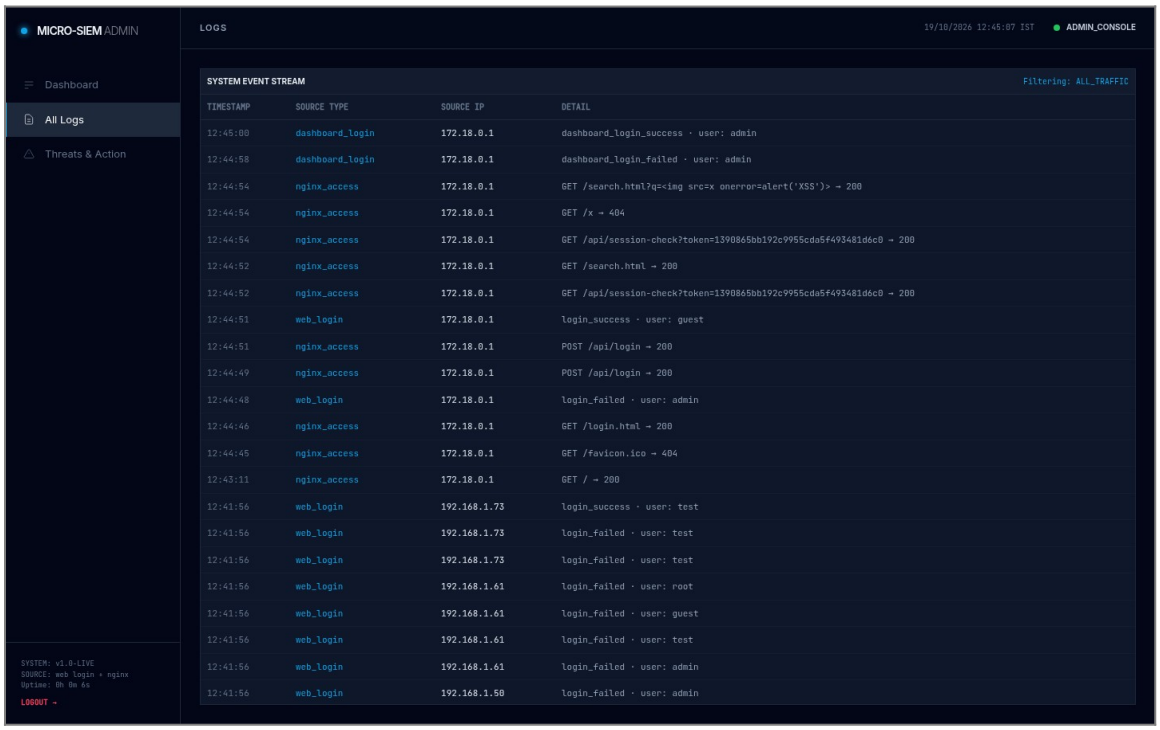


Figure 6.7: Admin Console – All Logs (system event stream with readable details)

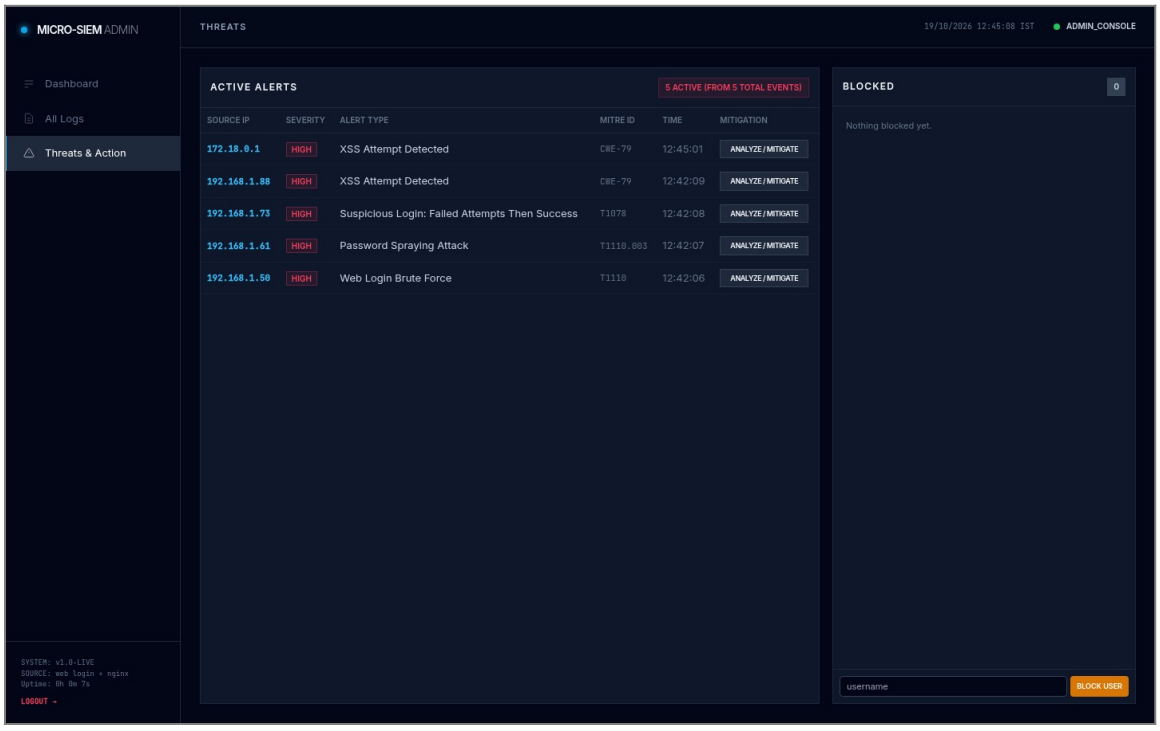


Figure 6.8: Admin Console – Threats & Action (MITRE-mapped alerts per source IP)

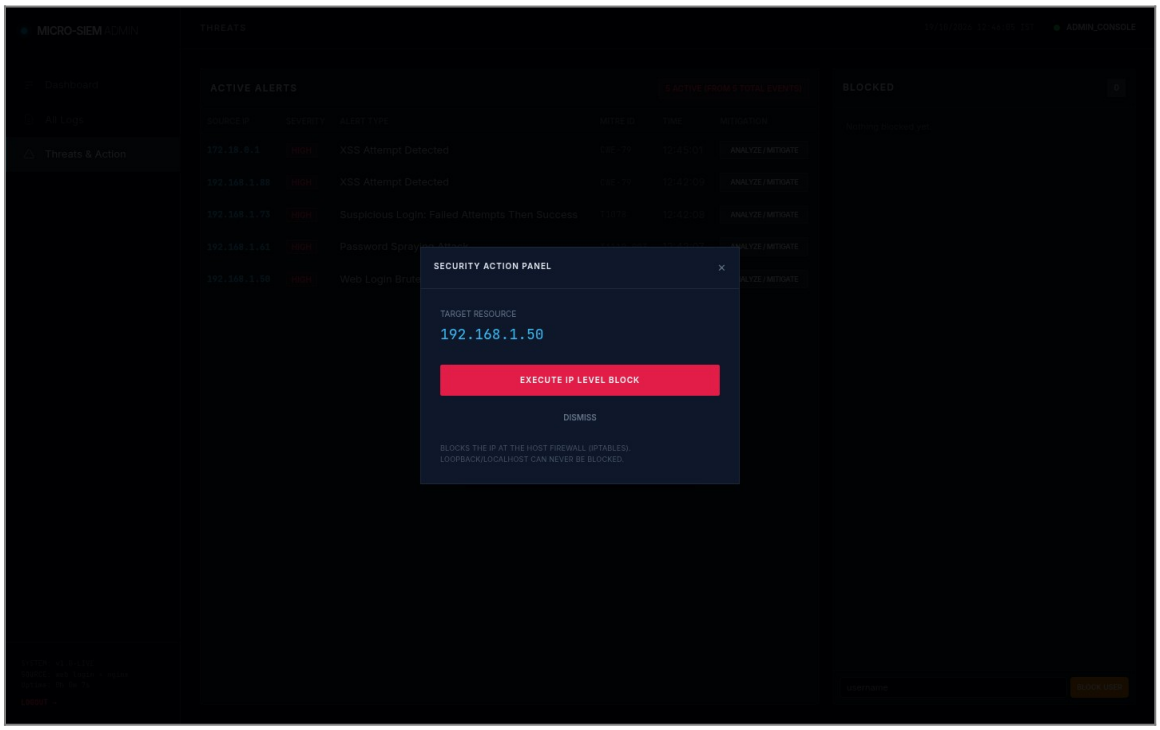


Figure 6.9: Admin Console – Security Action Panel for the brute-force attacker

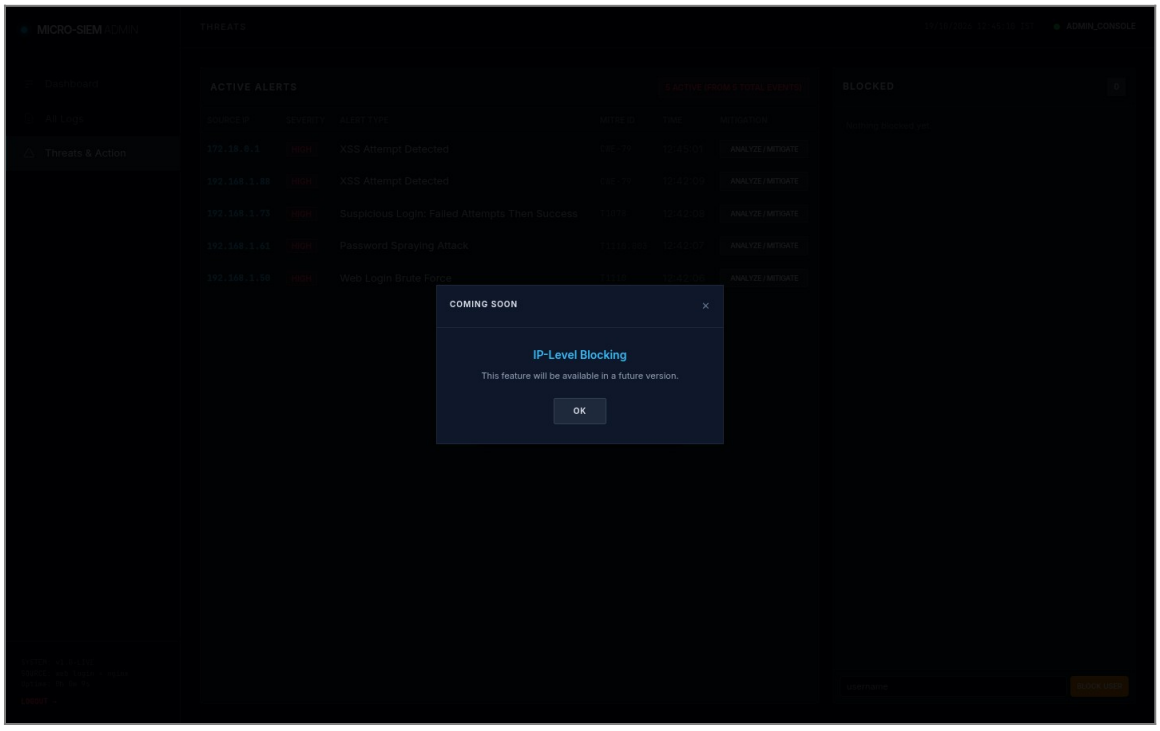


Figure 6.10: Admin Console – IP blocking shows "Coming Soon" (response module disabled)

MODULE NAME – VIEWER

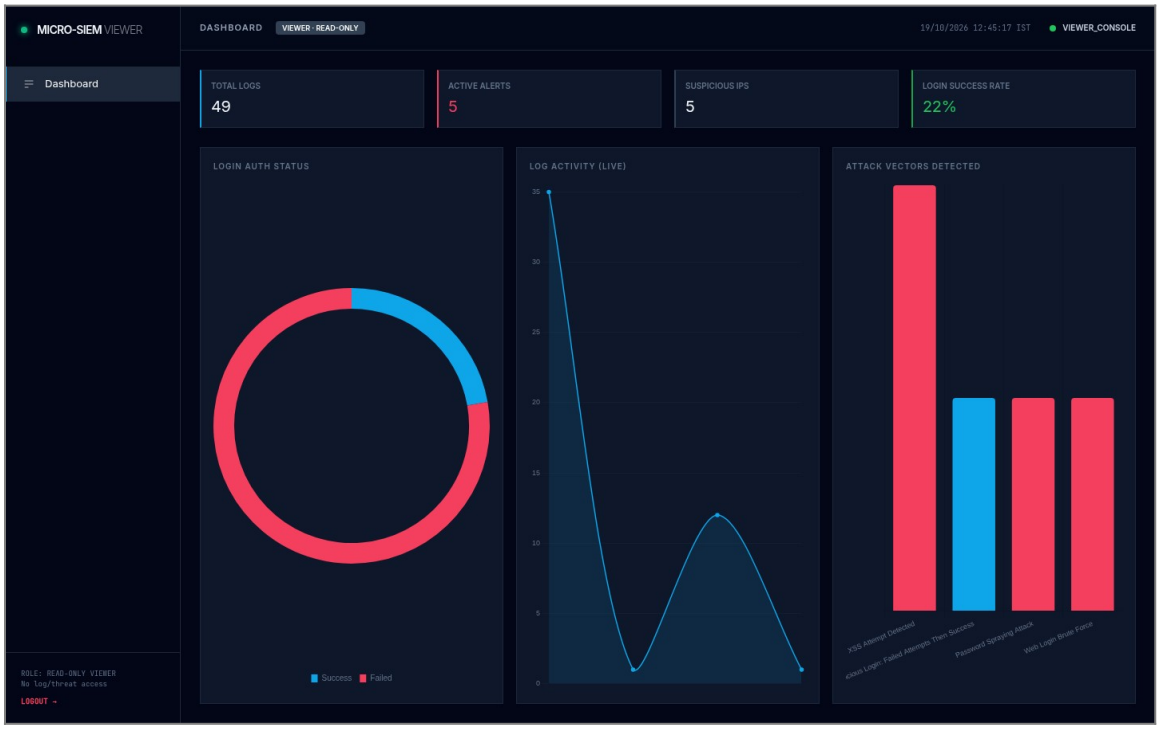


Figure 6.11: Viewer Dashboard – read-only statistics and charts

MODULE NAME – API AND DATA

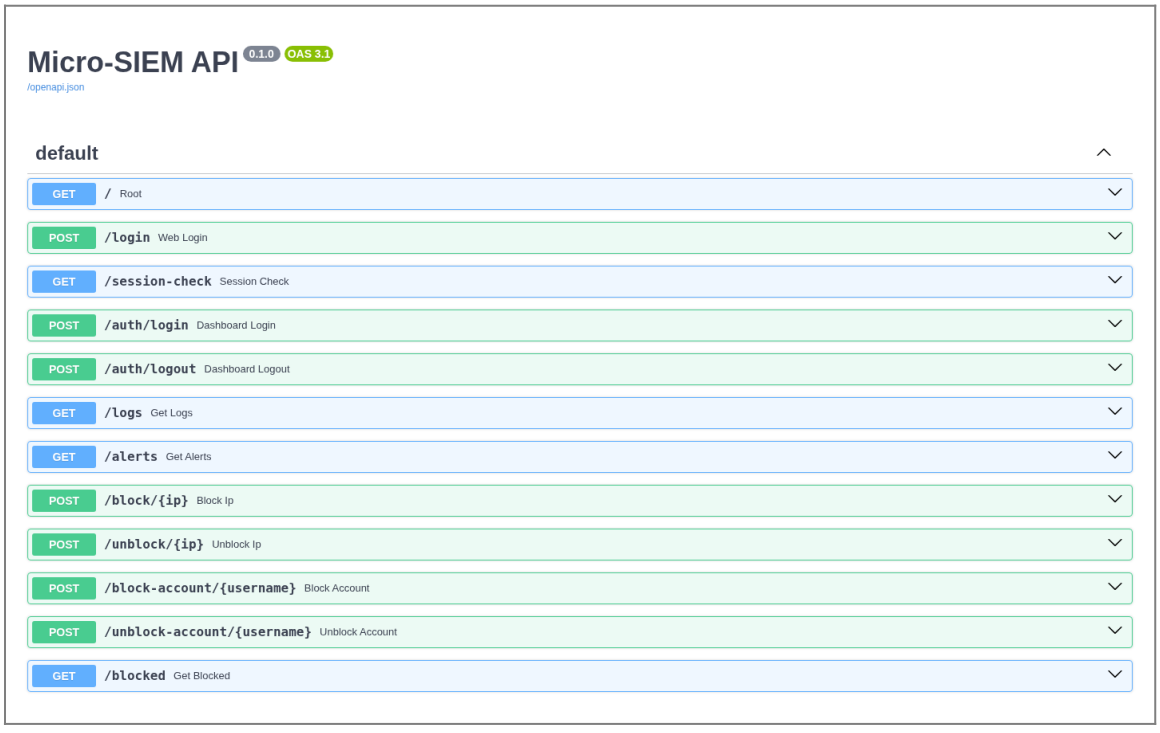


Figure 6.12: FastAPI Interactive Documentation – all 12 API endpoints

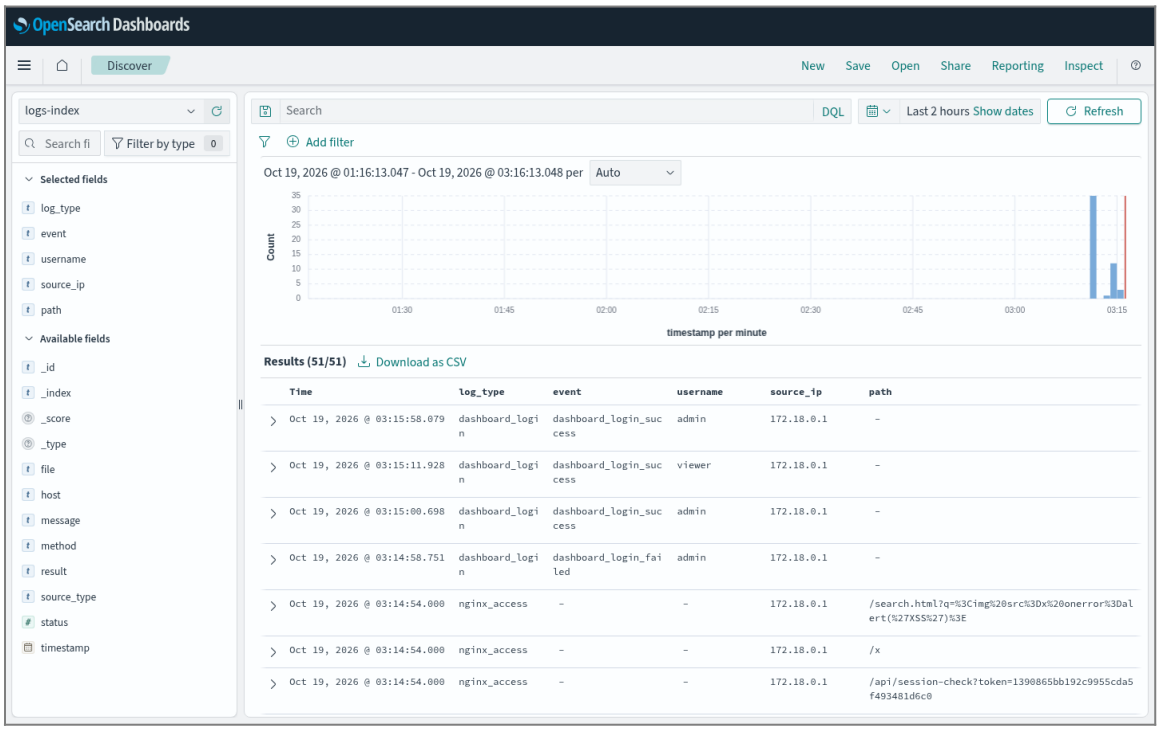


Figure 6.13: OpenSearch Dashboards – Discover view of logs-index

MODULE NAME – DEPLOYMENT AND LIVE DETECTION

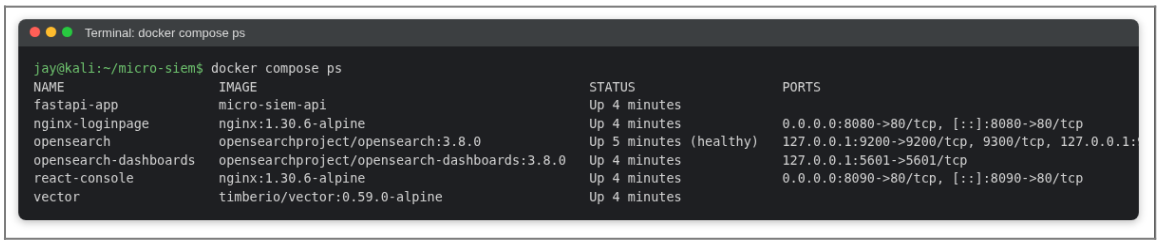


Figure 6.14: All six containers running (docker compose ps)

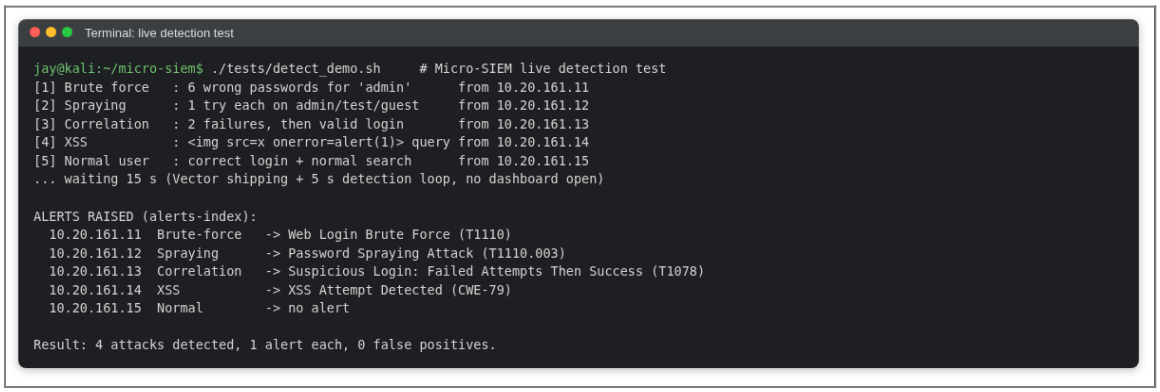


Figure 6.15: Live detection test – each attack raises exactly one alert, normal user raises none

7. SYSTEM TESTING

7.1 TESTING STRATEGIES

System testing is the phase in which the complete, integrated system is evaluated against its requirements. For a security monitoring system testing has two goals: the SIEM must **detect** every attack it is designed for without false alarms, and the SIEM must itself be **secure**. Micro-SIEM was tested offline with a mocked database, live on the running Docker stack with real attack traffic, and in a real (headless) web browser.

1. White-Box Testing

White-box (glass-box) testing examines the internal logic of the code. Each detection rule, the IP trust function, the rate limiter, the session handling and the firewall helper functions were tested directly with known inputs against a fake OpenSearch client (48 automated checks).

- All branches of every rule (threshold reached / not reached, already alerted / not alerted).
- Boundary values: exactly 1 vs. 2 failures before success, 2 vs. 3 usernames, case variants of usernames.
- Error paths: missing index, OpenSearch not yet ready, missing firewall chain.

2. Black-Box Testing

Black-box testing checks the behaviour from the outside without looking at the code. All 12 API endpoints were called with valid, invalid, missing and malicious input, and the HTTP status and response were compared with the expected result (49 checks).

- Incorrect or missing input (422), missing / invalid / expired token (401), wrong role (403), disabled feature (503).
- Boundary values for the limit parameter (0, 1000, 1001, 5000, non-numeric).

3. Unit Testing

- **Client IP module:** trusted proxy header accepted, HTML in header rejected, untrusted peer cannot spoof.
- **Protected addresses:** loopback in all forms (::1, 0:0::1, 127.0.0.2, ::ffff:127.0.0.1), own IP and Docker range refused.
- **Detection rules:** each rule fires only for its own pattern; XSS payloads detected even when URL-encoded or double-encoded.
- **Frontend helpers:** time conversion for both timestamp formats; HTML escaping of quotes and tags.

4. Integration Testing

Integration testing verifies that the components work together correctly across the whole pipeline.

1. **Ingestion pipeline:** demo login / Nginx request → log file → Vector → OpenSearch logs-index.

2. **Detection pipeline:** logs-index → background detection loop → alerts-index (with no dashboard open).
3. **Dashboard pipeline:** browser → Nginx /api proxy → FastAPI → OpenSearch → rendered charts and tables.
4. **Account blocking:** block via admin API → enforced on the real demo login and existing sessions → unblock.

5. System and Security Testing

- Live end-to-end run of all four attacks plus normal traffic from distinct client IPs.
- Security tests: rate limit, IP spoofing from an untrusted network, OpenSearch reachability from another network.
- Browser test: login, all dashboard views, modal and logout in headless Chromium with zero JavaScript errors.

7.2 TEST CASES

TC ID	Test Case	Input / Steps	Expected Result	Status
TC-01	Log ingestion	Demo login attempts and page requests	Events appear in logs-index with correct IP and timestamp	Pass
TC-02	Brute force	6 wrong passwords for 'admin' from one IP	1 'Web Login Brute Force' alert (T1110), no spraying alert	Pass
TC-03	Password spraying	1 wrong password each for admin, test, guest	1 'Password Spraying Attack' alert (T1110.003), no brute force	Pass
TC-04	Correlation	2 failures, then correct password	1 'Failed Attempts Then Success' alert (T1078)	Pass
TC-05	Correlation boundary	1 failure, then correct password	No alert	Pass
TC-06	XSS (encoded)	search.html?q=%3Cimg%20src%3Dx%20onerror%3D…%3E	1 'XSS Attempt Detected' alert (CWE-79)	Pass
TC-07	Normal user	Correct login + normal search	No alert (no false positive)	Pass
TC-08	Alert de-duplication	Attack continues after first alert	Still exactly 1 alert per rule per IP	Pass
TC-09	Detection without dashboard	Attack with no dashboard open	Alert raised by background loop	Pass
TC-10	Blocked-account attacks	6 attempts on a blocked account	Brute force alert (login_blocked counted)	Pass
TC-11	Dashboard login	Correct admin / viewer credentials	Token issued with correct role	Pass
TC-12	Invalid login	Wrong password / unknown user	HTTP 401, attempt logged	Pass
TC-13	Rate limiting	6 wrong dashboard logins from one IP	401 ×5 then 429 Too Many Requests	Pass
TC-14	Missing token	GET /alerts without token	HTTP 401	Pass
TC-15	Role check	Viewer calls POST /block/8.8.8.8	HTTP 403 Forbidden	Pass
TC-16	Limit validation	GET /logs?limit=99999	HTTP 422 (not 500)	Pass
TC-17	Logout	POST /auth/logout, reuse token	Token rejected with 401	Pass
TC-18	IP spoofing (direct)	Fake X-Real-IP from untrusted network	Real peer IP logged, fake ignored	Pass
TC-19	Stored XSS via header	X-Forwarded-For: <img src=x onerror=…>	Value rejected, never stored or rendered	Pass
TC-20	Database exposure	Connect to :9200 from another network	Connection refused; works on localhost only	Pass
TC-21	Account blocking	Admin blocks 'test', user logs in	Refused even with right password; session ended	Pass
TC-22	Account unblocking	Admin unblocks 'test'	User can log in immediately	Pass
TC-23	Dashboard rendering	Login in headless Chromium, open all views	Charts, tables, IST times; 0 JS errors	Pass
TC-24	Readable log details	Open All Logs view	No raw JSON; XSS payload shown as text only	Pass

TC ID	Test Case	Input / Steps	Expected Result	Status
TC-25	Fresh start	Start stack with empty indices	Endpoints return 200 with empty lists	Pass

Summary of Test Results

Test Suite	Environment	Checks	Result
Unit / white-box (mocked OpenSearch)	Python virtual environment	48	48 passed
Live pipelines (ingestion, detection, dashboards, security)	Docker stack	35	34 passed, 1 inconclusive*
All 12 API endpoints (black-box)	Docker stack	49	49 passed
Account blocking end-to-end	Docker stack	28	28 passed
Attacks on blocked accounts	Docker stack	7	7 passed
Dashboards in a real browser	Headless Chromium	UI flows	0 JavaScript errors

* Spoofing a valid-looking IP from a second physical device on the Wi-Fi network cannot be simulated from one machine and is verified manually with a mobile phone.

8. CONCLUSION AND FUTURE SCOPE

Conclusion:

Micro-SIEM successfully demonstrates the complete working of a Security Information and Event Management system using only free and open-source tools. Web login events and Nginx access logs are collected, shipped by Vector, indexed by OpenSearch and analysed automatically every five seconds. Four detection rules recognise brute-force, password-spraying, credential-compromise and Cross-Site Scripting attacks, and every alert is mapped to the MITRE ATT&CK framework or to OWASP/CWE. Admin and viewer dashboards present the security situation clearly, while server-side sessions, roles, rate limiting and validated input keep the monitoring system itself secure.

Extensive testing (unit, API, integration, live attack scenarios and browser tests) showed that every attack produces exactly one correctly classified alert and normal users produce none. The project gave practical experience of the tools and ideas used in real Security Operations Centres.

Key benefits of the system:

1. **Centralized visibility:** All security events in one searchable place.
2. **Real-time detection:** Attacks detected within seconds, even with no one watching.
3. **Standard mapping:** MITRE ATT&CK and CWE identifiers make alerts meaningful.
4. **Secure by design:** The SIEM resists IP spoofing, stored XSS and brute force against itself.
5. **Low cost and portable:** Runs with one command on a single laptop, even offline.

Limitations:

- Only two log sources (web application and Nginx) are monitored in this version.
- Automatic blocking is implemented but disabled; responses are manual.
- OpenSearch runs as a single node without its security plugin (protected by binding to localhost).
- Rules are threshold and signature based, so new or slow attacks may go unnoticed.

Future Scope:

1. **Enable the response module:** Activate IP blocking (INPUT + DOCKER-USER chains) and account blocking after field testing.
2. **More log sources:** SSH / Linux auth logs, Windows Event Logs, firewall and DNS logs.
3. **Sigma rule support:** Import community detection rules written in the vendor-neutral Sigma format.
4. **Additional rules:** Web scanning / directory enumeration, impossible travel, privilege escalation.
5. **Alert notifications:** E-mail, Telegram or Slack messages for high-severity alerts.

6. **Threat intelligence and GeoIP:** Enrich IPs with reputation, country and ASN information.
7. **Machine learning:** Behaviour-based anomaly detection for unknown attack patterns.
8. **Production hardening:** OpenSearch security plugin with TLS, index lifecycle management for retention, multi-node cluster.
9. **Incident management:** Alert status workflow (new, investigating, resolved) with analyst notes.
10. **Cloud deployment:** Hosting the stack on a cloud VM for continuous monitoring.

9. BIBLIOGRAPHY

Category	Details
Books	1. Security Information and Event Management (SIEM) Implementation – David R. Miller, Shon Harris, Allen Harper, Stephen VanDyke, Chris Blask (McGraw-Hill) 2. Applied Network Security Monitoring – Chris Sanders, Jason Smith (Syngress) 3. The Practice of Network Security Monitoring – Richard Bejtlich (No Starch Press) 4. The Web Application Hacker's Handbook – Dafydd Stuttard, Marcus Pinto (Wiley) 5. Docker Deep Dive – Nigel Poulton
Standards & Research Papers	1. NIST SP 800-92: Guide to Computer Security Log Management – K. Kent, M. Souppaya 2. NIST SP 800-61 Rev. 2: Computer Security Incident Handling Guide 3. MITRE ATT&CK: Design and Philosophy – The MITRE Corporation 4. OWASP Top 10 – Open Worldwide Application Security Project
Websites	1. https://attack.mitre.org – MITRE ATT&CK techniques T1110, T1110.003, T1078 2. https://cwe.mitre.org/data/definitions/79.html – CWE-79 Cross-Site Scripting 3. https://owasp.org – OWASP XSS prevention and Top 10 4. https://opensearch.org/docs – OpenSearch and OpenSearch Dashboards documentation 5. https://vector.dev/docs – Vector sources, VRL and sinks 6. https://fastapi.tiangolo.com – FastAPI documentation 7. https://nginx.org/en/docs – Nginx log_format and real_ip module 8. https://docs.docker.com – Docker Compose, networking and packet filtering (DOCKER-USER) 9. https://www.chartjs.org and https://tailwindcss.com – frontend libraries
College Materials	1. Software engineering and SDLC notes from faculty 2. Lecture notes on requirement analysis, UML and system design
Tools & Technologies	Backend: Python, FastAPI, Uvicorn Storage: OpenSearch, OpenSearch Dashboards Log pipeline: Vector Web server: Nginx Frontend: HTML, Tailwind CSS, JavaScript, Chart.js Deployment: Docker, Docker Compose Diagrams: Graphviz, Matplotlib Version control: Git, GitHub